HomeMade Pickles & Snacks: Taste the Best – A Cloud-Based Platform for Ordering Authentic Homemade Pickles and Snacks

Project Description:

Taste the Best is a cloud-based culinary platform revolutionizing access to authentic, handcrafted pickles and snacks. Addressing the growing demand for preservative-free, traditional recipes, this initiative combines artisanal craftsmanship with cutting-edge technology to deliver farm-fresh flavors directly to consumers. Built on Flask for backend efficiency and hosted on AWS EC2 for scalable performance, the platform offers seamless browsing, ordering, and subscription management. DynamoDB ensures real-time inventory tracking and personalized user experiences, while fostering sustainability through partnerships with local farmers and eco-friendly packaging. From tangy regional pickles to wholesome snacks, every product celebrates heritage recipes, nutritional integrity, and convenience - proving that tradition and innovation can coexist deliciously. "Preserving Traditions, One Jar at a Time."

Scenario 1: Scalable Order Management for High Demand

A cloud-based system ensures seamless order processing during peak user activity. For instance, during a promotional event, hundreds of users simultaneously access the platform to place orders. The backend efficiently processes requests, updates inventory in real-time, and manages user sessions. The cloud infrastructure handles traffic spikes without performance degradation, ensuring smooth transactions and minimizing wait times.

Scenario 2: Real-Time Inventory Tracking and Updates

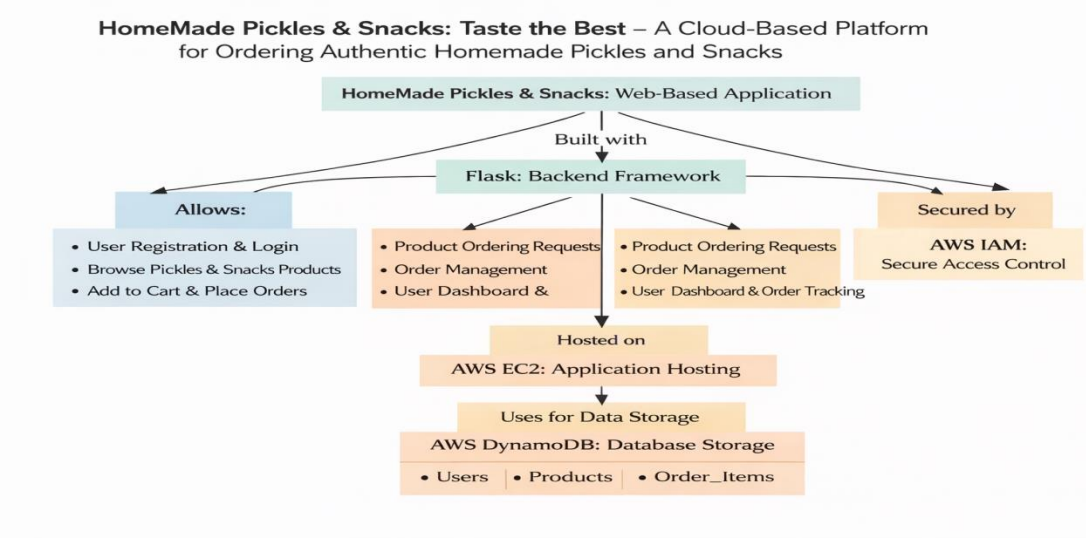
When a customer places an order for a product, the system instantly updates stock levels and records transaction details. For example, a user purchases an item, triggering automatic inventory deduction and order confirmation. Staff members receive updated dashboards to monitor stock availability and fulfillment progress, ensuring timely restocking and minimizing overselling risks.

Scenario 3: Personalized User Experience and Recommendations

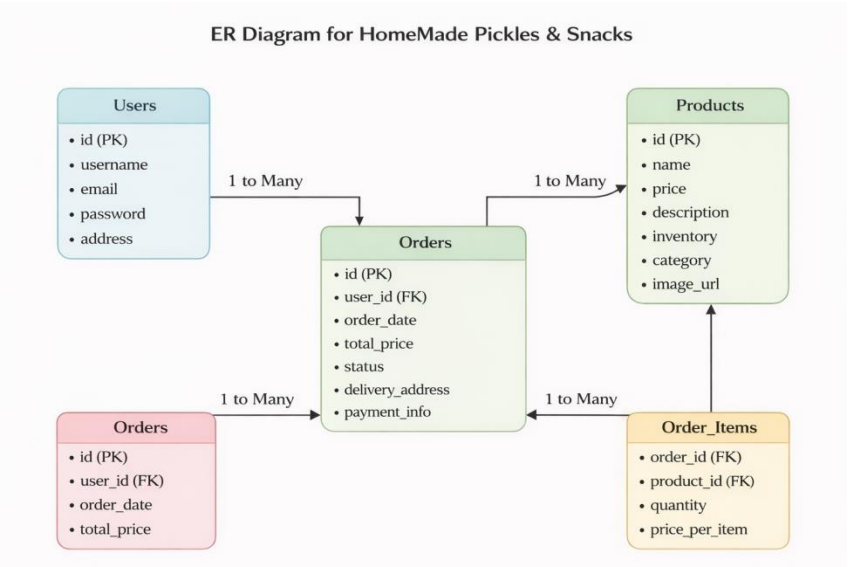
The platform leverages user behavior data to enhance engagement. A returning customer, for instance, views tailored recommendations based on past purchases and browsing history. The system dynamically adjusts suggestions in real-time, while maintaining fast response rates even during high traffic, creating a frictionless and intuitive shopping experience.

.

AWS ARCHITECTURE



Entity Relationship (ER)Diagram:



Pre-requisites:

1. **.AWS Account Setup**: AWS Account Setup
2. **Understanding IAM**: IAM Overview
3. **Amazon EC2 Basics**: EC2 Tutorial
4. **DynamoDB Basics**: DynamoDB Introduction
5. **Git Version Control**: Git Documentation

Project WorkFlow:

1. AWS Account Setup and Login

Activity 1.1: Set up an AWS account if not already done.

Activity 1.2: Log in to the AWS Management Console

2. DynamoDB Database Creation and Setup

Activity 2.1: Create a DynamoDB Table.

Activity 2.2: Configure Attributes for travel-Users and Bookings Tables.

3. Backend Development and Application Setup

Activity 3.1:Develop the Backend Using Flask.

4. IAM Role Setup

Activity 4.1: Create IAM Role

Activity 4.2: Attach Policies

5. EC2 Instance Setup

Activity 5.1: Launch an EC2 instance to host the Flask application.

Activity 5.2: Configure security groups for HTTP, and SSH access.

6. Deployment on EC2

Activity 6.1:Upload Flask Files

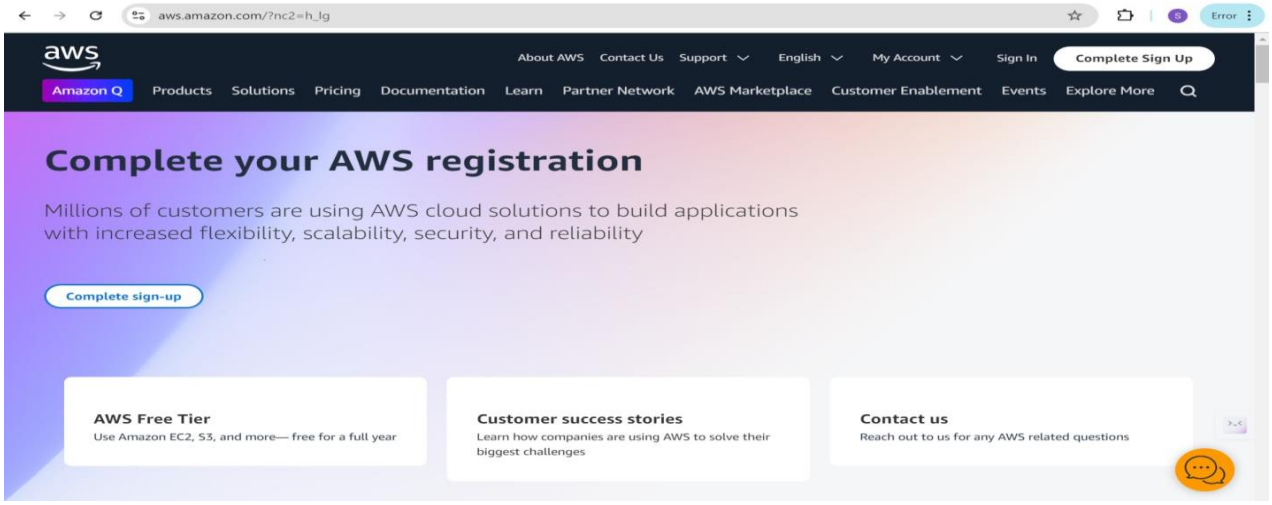
Activity 6.2: Run the Flask App

7. Testing and Deployment

Activity 7.1: Conduct functional testing to verify user registration, login, book requests, and notifications.

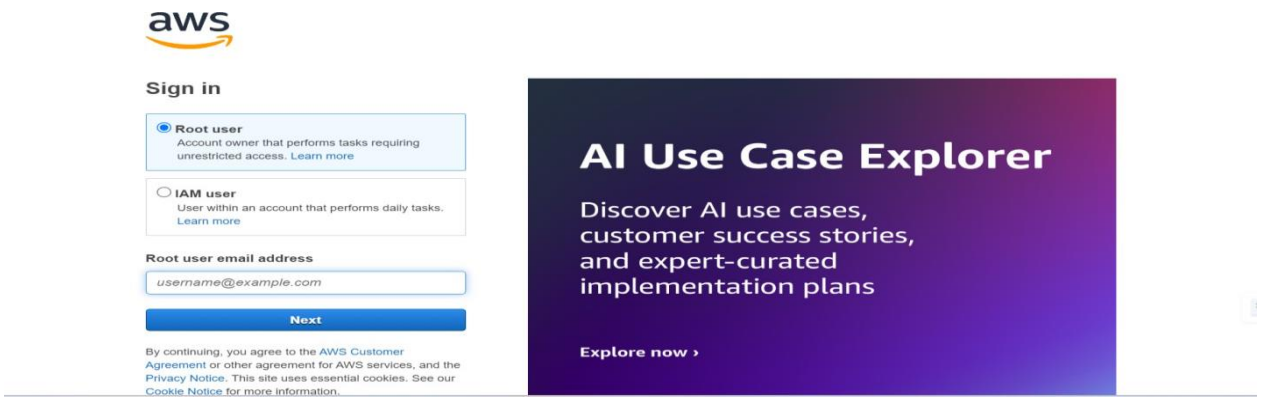
Milestone 1: AWS Account Setup and Login

- **Activity 1.1: Set up an AWS account if not already done.**
  - Sign up for an AWS account and configure billing settings.



- **Activity 1.2: Log in to the AWS Management Console**

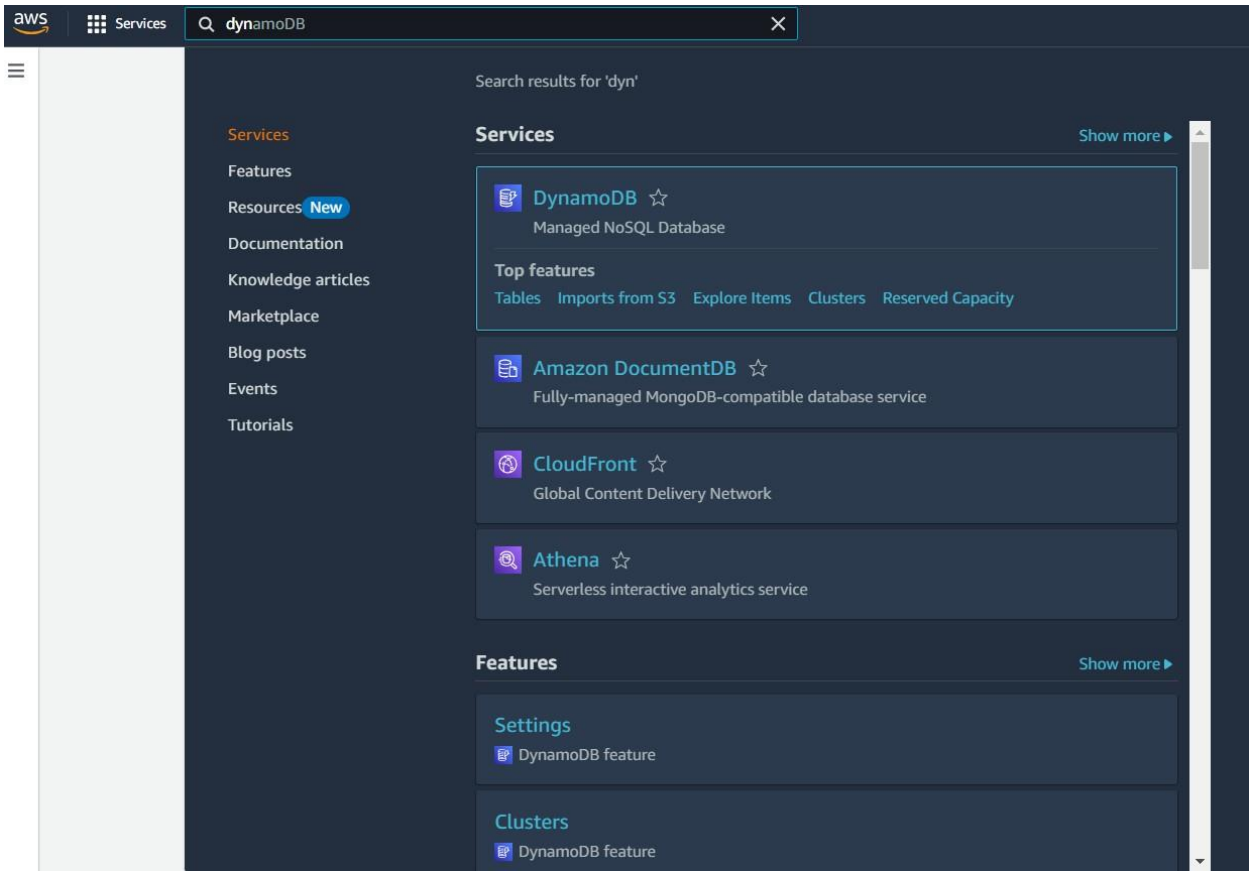
  - After setting up your account, log in to the AWS Management Console.

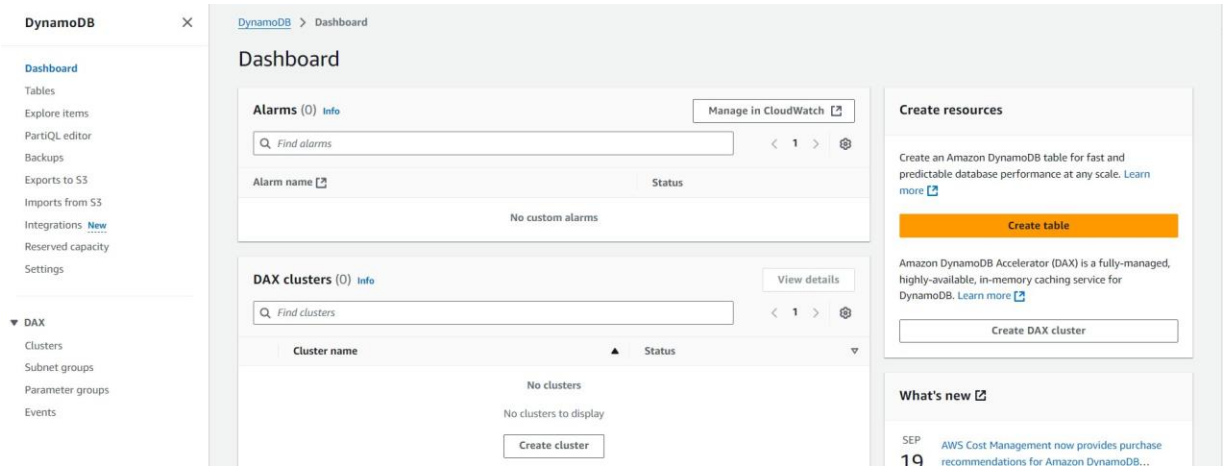


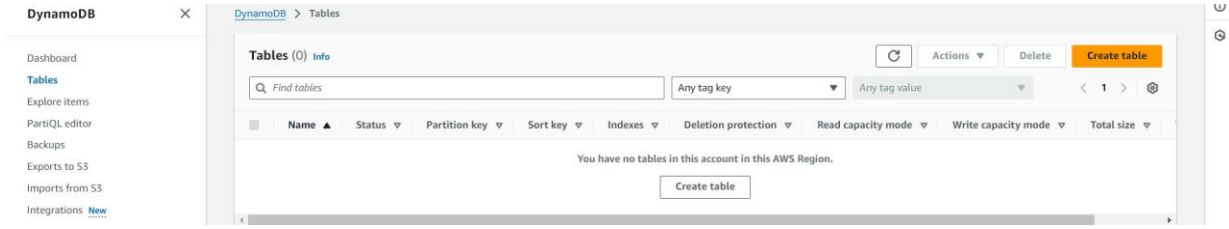
Milestone 2: DynamoDB Database Creation and Setup

- **Activity 2.1:Navigate to the DynamoDB**

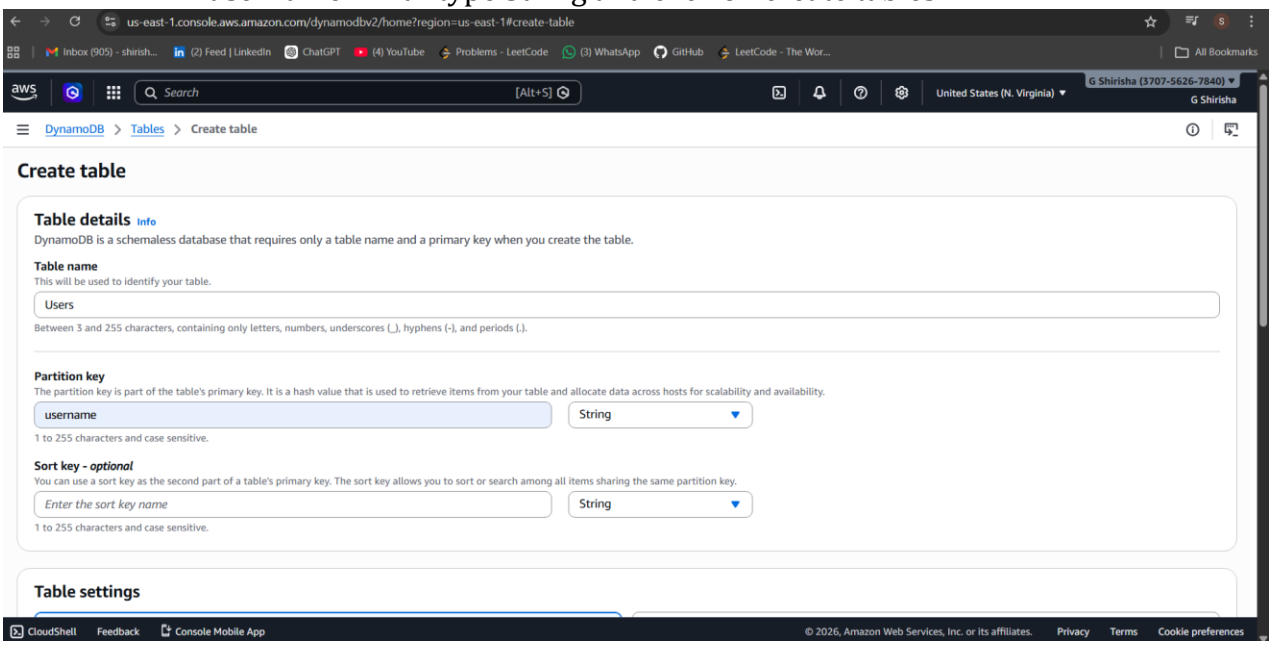
- In the AWS Console, navigate to DynamoDB and click on create tables.



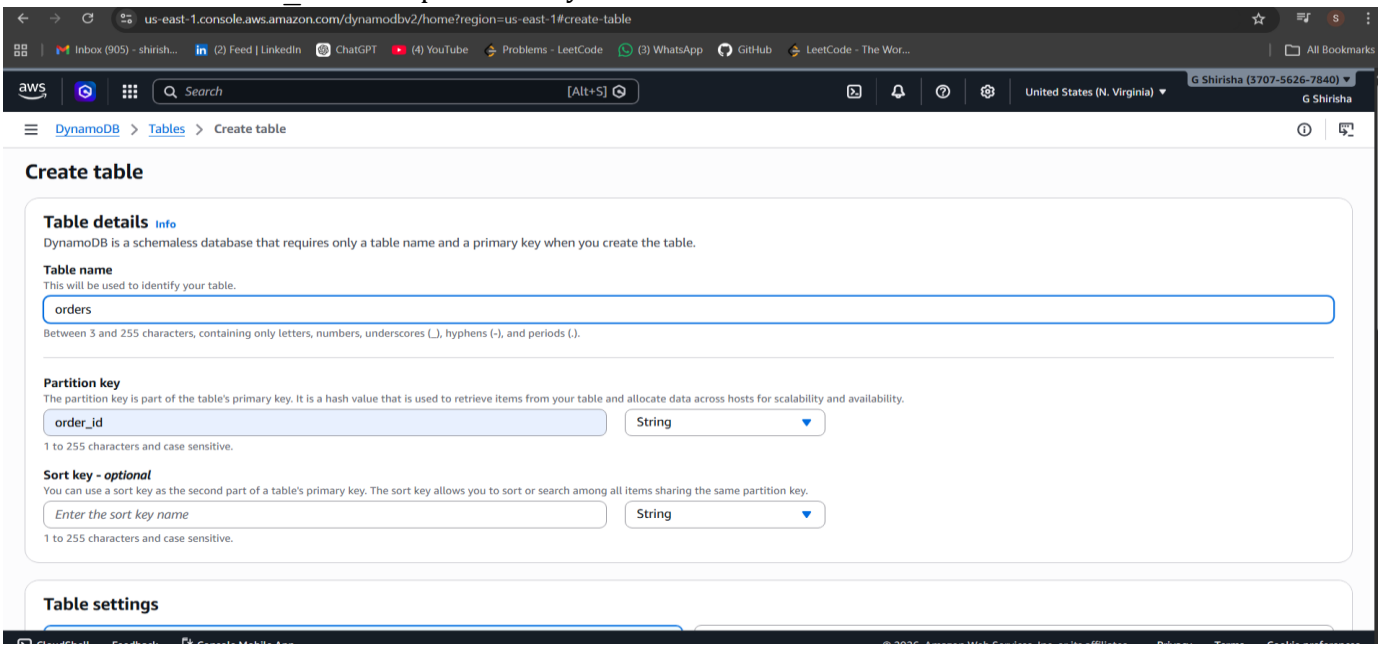


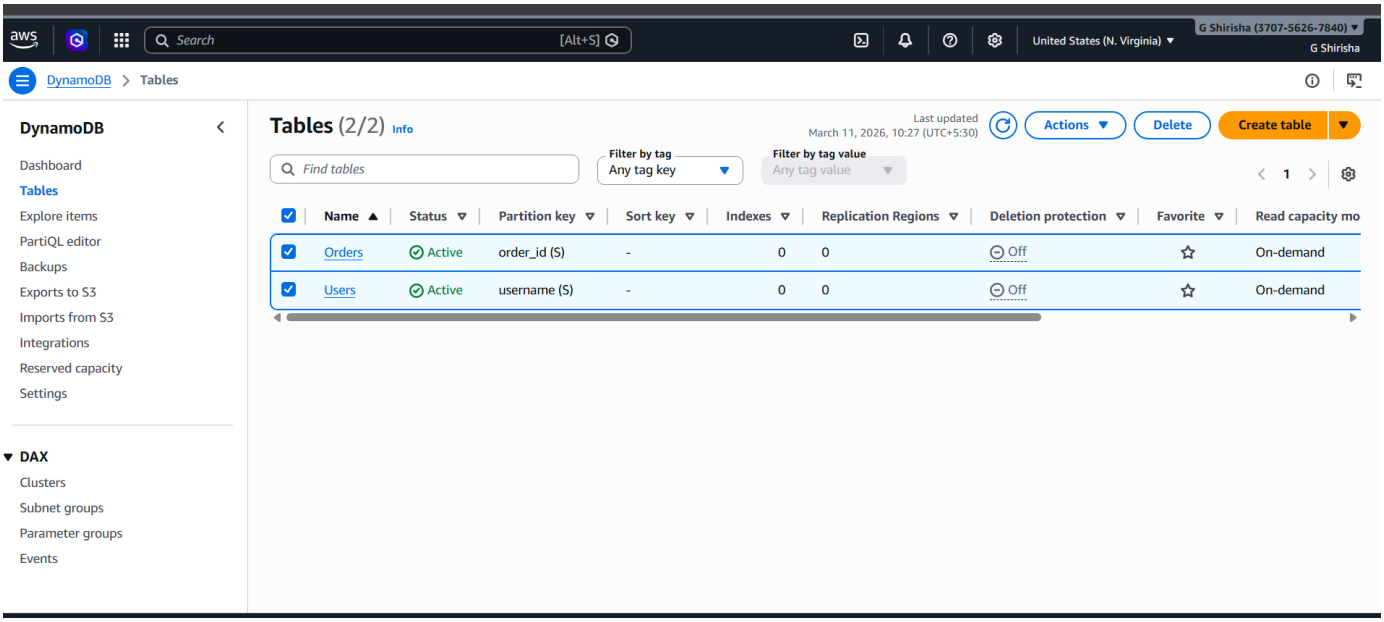


- **Activity 2.2:Create DynamoDB tables for storing registration details and booking records.**
    - Create a Users table for storing registered user details with partition key "username" with type String and click on create tables.



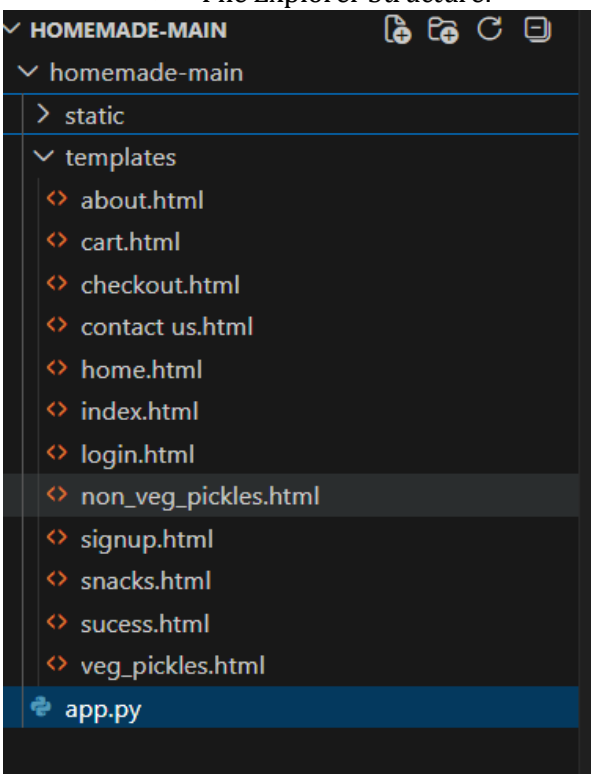
- Follow the same steps to create a `orders` for storing order records with `Order_id` as the partition key.





Milestone 3:Backend Development and Application Setup

- **Activity 3.1: Develop the backend using Flask**
  - File Explorer Structure:



Description: **Create a** templates folder **containing HTML files for the main features of the *HomeMade Pickles & Snacks* platform such as** home page, product listing, cart, order placement, registration, login, and dashboard**. The backend logic and routing are managed in** app.py**, which handles user authentication, product management, and order processing.**

- **Flask App Initialization**

  Import Libraries:

```
from flask import Flask, render_template, request, redirect, session
import uuid
import boto3
import os
from decimal import Decimal
from boto3.dynamodb.conditions import Attr
from utils.data import transport_data, hotel_data
```

Description: Import necessary Flask modules for web routing, Boto3 for AWS DynamoDB access, UUID for unique IDs, and custom transport and hotel data from `utils.data`

Initialize flask app:

```
app = Flask(__name__)
```

Description: Initialize the Flask application by creating an instance of the `Flask` class with the current module name.

DynamoDB Connection:

```
app.secret_key = 'your_very_secret_key_12345'

dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')
users_table = dynamodb.Table('Users')
orders_table = dynamodb.Table('Orders')
```

Description: Use **boto3** to connect to **DynamoDB** for users,Orders database operations and also mention region_name where Dynamodb tables are created.

- **Routes for Web Pages**

Register User:

```python
@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        username = request.form['username'].strip()
        email = request.form['email'].strip()
        password = request.form['password']

        try:
            # Check if username exists
            response = users_table.get_item(Key={'username': username})
            if 'Item' in response:
                return render_template('signup.html', error='Username already exists')

            # Hash password before storing
            hashed_password = generate_password_hash(password)

            # Store new user in DynamoDB
            users_table.put_item(
                Item={
                    'username': username,
                    'email': email,
                    'password': hashed_password  # Store hashed password
                }
            )

            return redirect(url_for('login'))

        except Exception as e:
            app.logger.error(f"Signup error: {str(e)}")
            return render_template('signup.html', error='Registration failed. Please try again.')

    return render_template('signup.html')
```

Description: Collecting registration data, and stores user details in the database.

- **login Route (GET/POST)**:

```python
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        try:
            # Fetch user from DynamoDB
            response = users_table.get_item(Key={'username': username})

            if 'Item' not in response:
                return render_template('login.html', error='User not found')

            user = response['Item']

            # Ensure password field exists in the DB
            if 'password' not in user:
                return render_template('login.html', error='Password not found in database')

            # Verify password
            if check_password_hash(user['password'], password):
                session['logged_in'] = True
                session['username'] = username
                session.setdefault('cart', [])  # Initialize cart if not set
                return redirect(url_for('home'))

            return render_template('login.html', error='Invalid password')

        except Exception as e:
            app.logger.error(f"Database error: {str(e)}")
            return render_template('login.html', error='Login failed. Try again later')

    return render_template('login.html')
```

Description: Verifies user credentials, increments login count, and redirects to the dashboard on success.

Menu Routes:

```python
@app.route('/cart')
def cart():
    if not session.get('logged_in'):
        return redirect(url_for('login'))
    return render_template('cart.html')

@app.route('/checkout', methods=['GET', 'POST'])
def checkout():
    if not session.get('logged_in'):
        return redirect(url_for('login'))

    error_message = None  # Variable to hold error messages

    if request.method == 'POST':
        try:
            # Extract form data safely
            name = request.form.get('name', '').strip()
            address = request.form.get('address', '').strip()
            phone = request.form.get('phone', '').strip()
            payment_method = request.form.get('payment', '').strip()

            # Validate inputs
            if not all([name, address, phone, payment_method]):
                return render_template('checkout.html', error="All fields are required.")

            if not phone.isdigit() or len(phone) != 10:
                return render_template('checkout.html', error="Phone number must be exactly 10 digits.")

            # Get cart data from hidden inputs
            cart_data = request.form.get('cart_data', '[]')
            total_amount = request.form.get('total_amount', '0')

            try:
                cart_items = json.loads(cart_data)
                total_amount = Decimal(total_amount)  # ✅ Convert to Decimal for DynamoDB
            except (json.JSONDecodeError, ValueError):
                return render_template('checkout.html', error="Invalid cart data format.")

            # Ensure cart is not empty
            if not cart_items:
                return render_template('checkout.html', error="Your cart is empty.")
# ================= PRODUCT PAGE ROUTES =================
@app.route('/home')
def home():
    if not session.get('logged_in'):
        return redirect(url_for('login'))
    return render_template('home.html')

@app.route('/non_veg_pickles')
def non_veg_pickles():
    if not session.get('logged_in'):
        return redirect(url_for('login'))

    return render_template('non_veg_pickles.html', products=products['non_veg_pickles'])

@app.route('/veg_pickles')
def veg_pickles():
    if not session.get('logged_in'):
        return redirect(url_for('login'))

    # Simply pass all products without filtering
    return render_template('veg_pickles.html', products=products['veg_pickles'])

@app.route('/snacks')
def snacks():
    if not session.get('logged_in'):
        return redirect(url_for('login'))

    return render_template('snacks.html', products=products['snacks'])
```

Logout route:

```
@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))
```

Description: Implement the logout route to clear session data and reset `loggedIn` in localStorage before redirecting the user to the homepage.

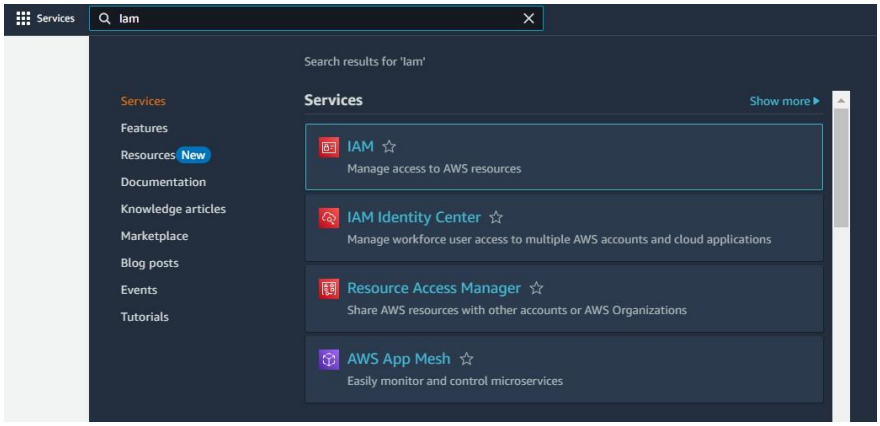
Deployment Code:

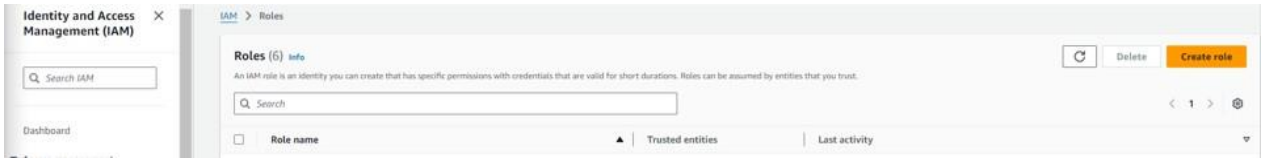
```
if __name__ == "__main__":
    app.run(host='0.0.0.0', port=80, debug=True)
```

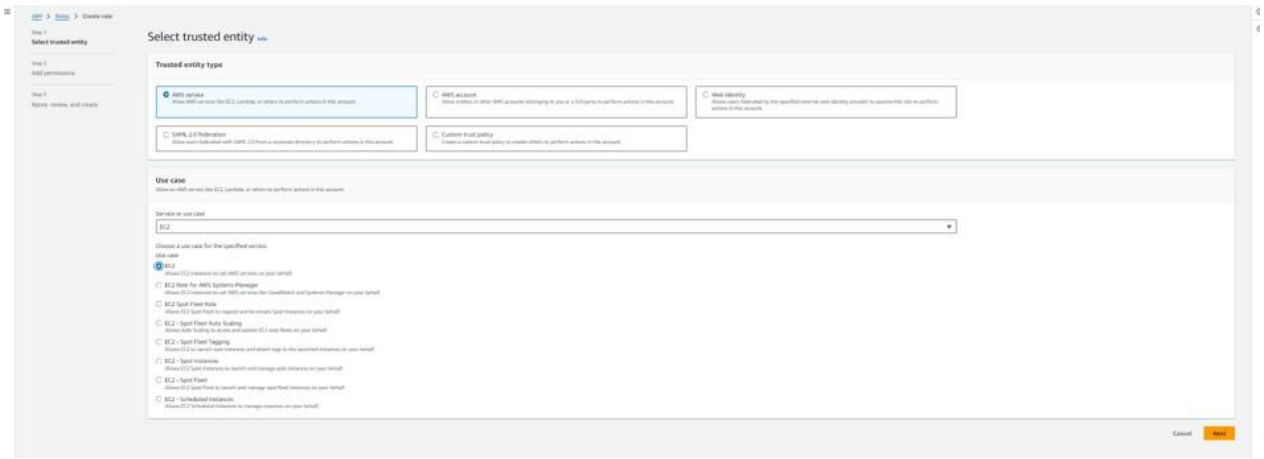
Description: Start the Flask application on all network interfaces at port 80 with debug mode enabled for easier development and testing.

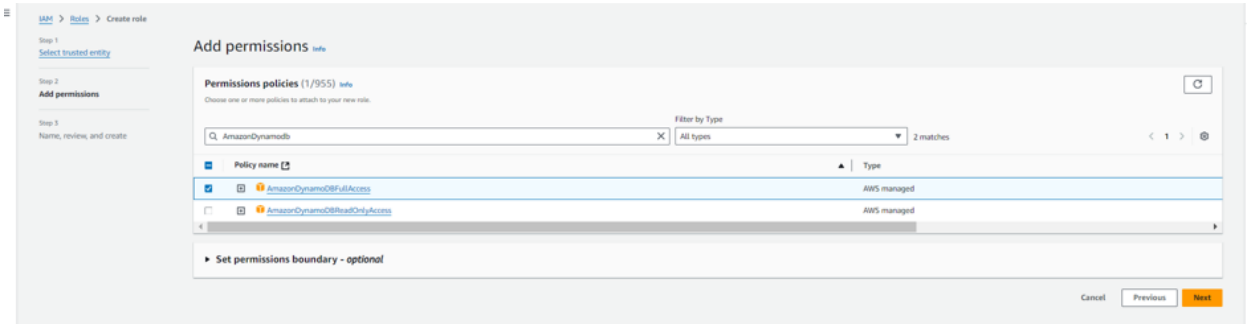
Milestone 4: IAM Role Setup

- **Activity 4.1:Create IAM Role.**
    - In the AWS Console, go to IAM and create a new IAM Role for EC2 to interact with DynamoDB and SNS.





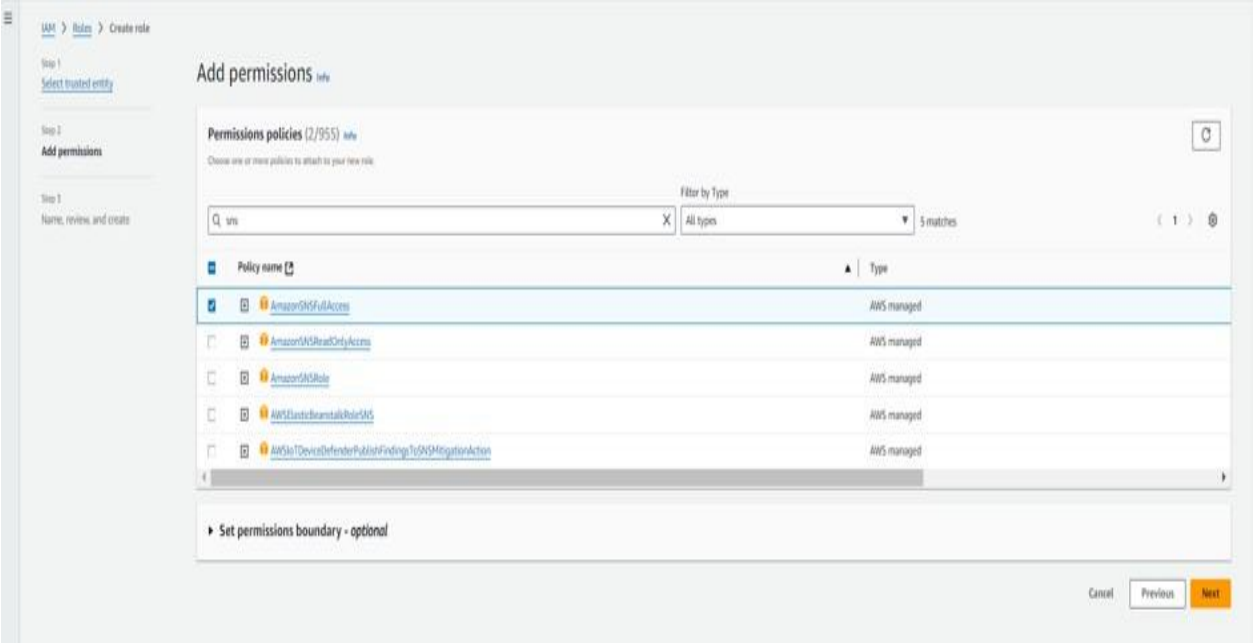


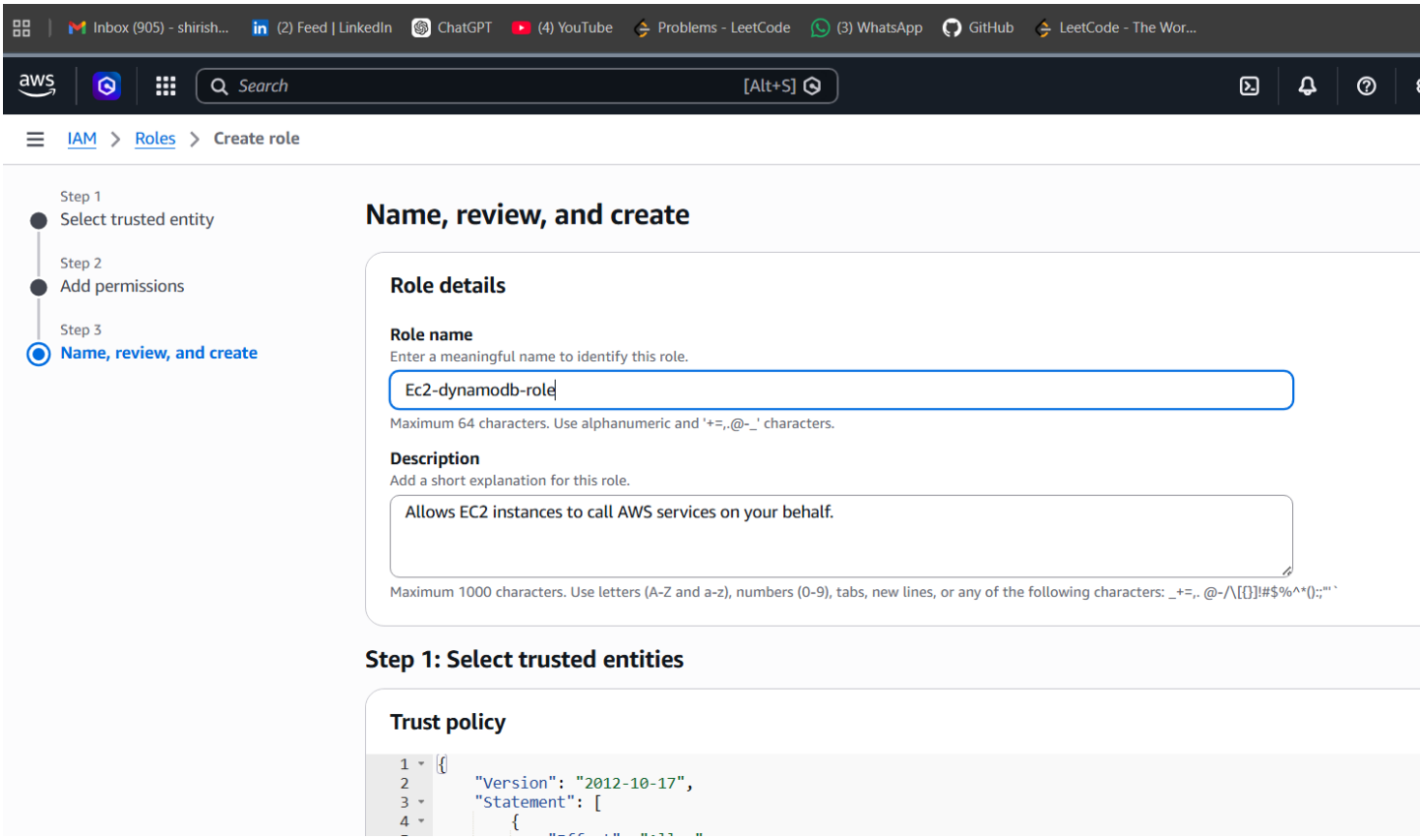


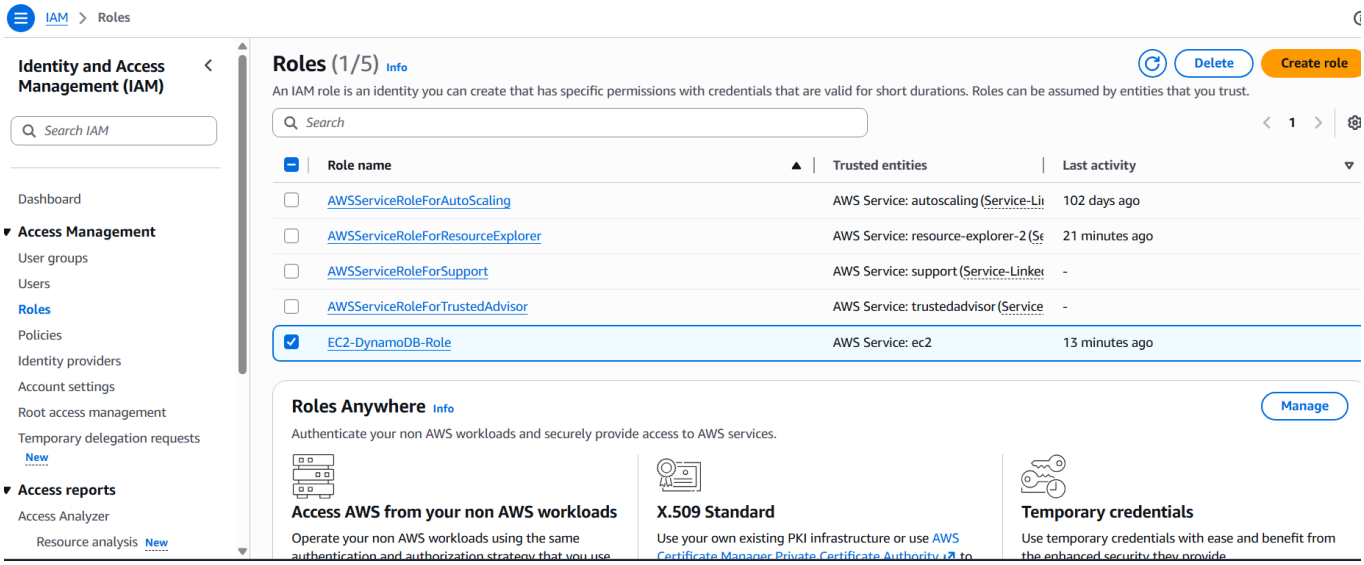
- **Activity 4.2: Attach Policies.**

  Attach the following policies to the role:

  - AmazonDynamoDBFullAccess: Allows EC2 to perform read/write operations on DynamoDB.

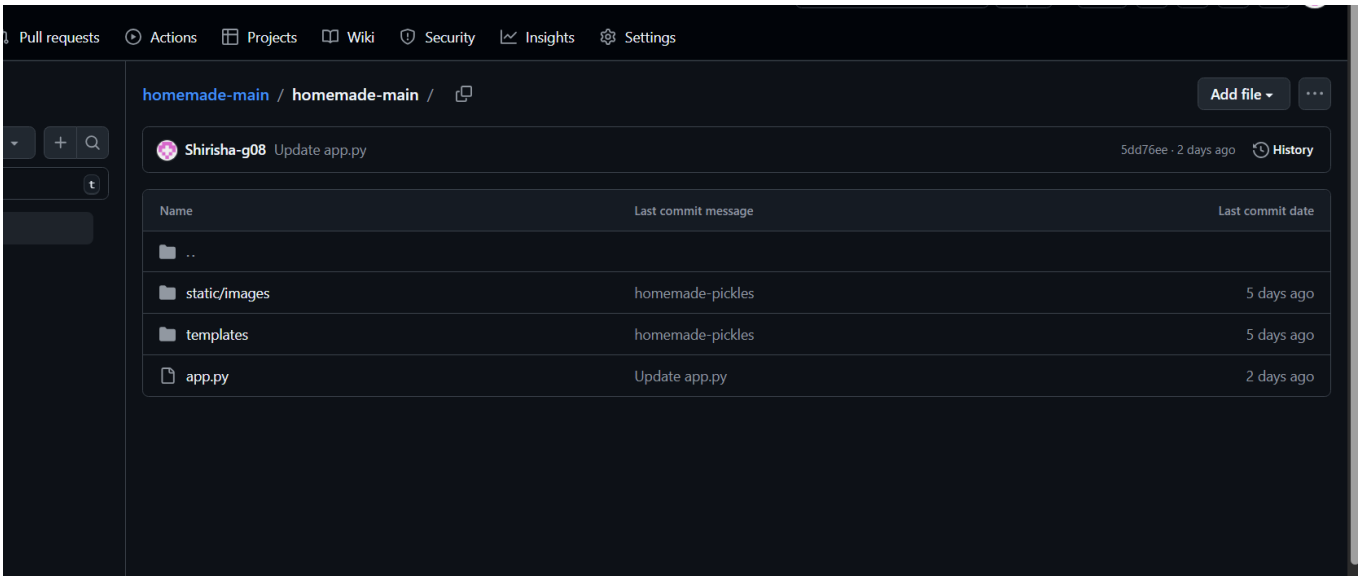

IAM > Roles > Create role
Add permissions
Permissions policies (2/955)
Filter by Type
All types
5 matches
Policy name
Type
AmazonSNSFullAccess
AWS managed
Set permissions boundary - optional
Cancel
Previous
Next


Inbox (905) - shirish...
(2) Feed | LinkedIn
ChatGPT
(4) YouTube
Problems - LeetCode
(3) WhatsApp
GitHub
LeetCode - The Wor...
aws
Search
[Alt+S]
IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Name, review, and create
Role details
Role name
Enter a meaningful name to identify this role.
Ec2-dynamodb-role
Maximum 64 characters. Use alphanumeric and '+=,.@-_' characters.
Description
Add a short explanation for this role.
Allows EC2 instances to call AWS services on your behalf.
Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=,. @-/\[{}]!#$%^*();:"'`
Step 1: Select trusted entities
Trust policy
{
"Version": "2012-10-17",
"Statement": [
{



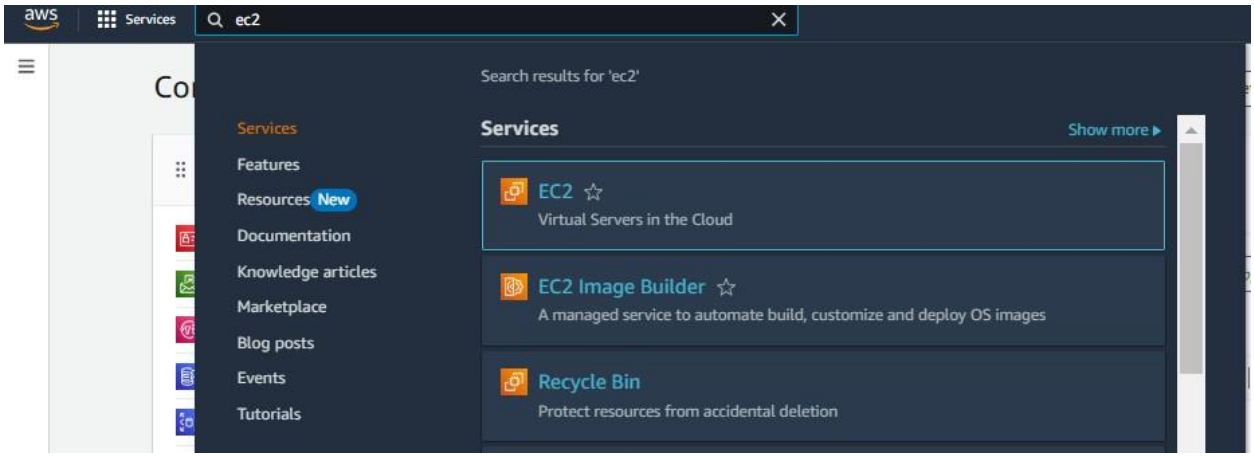
Milestone 5: EC2 Instance Setup

- **Note: Load your Flask app and Html files into GitHub repository.**

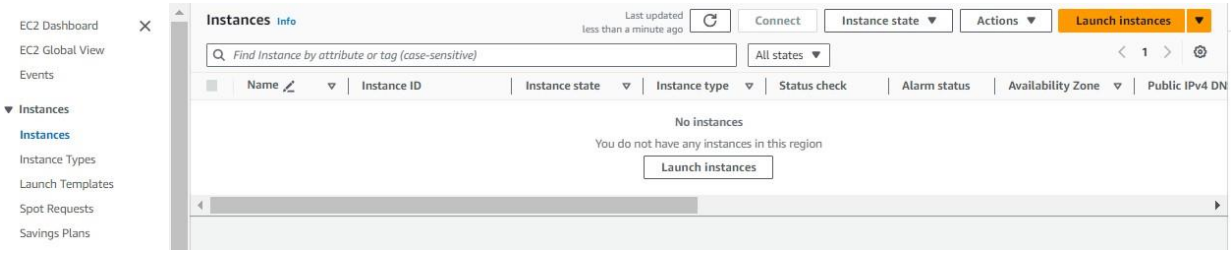


- **Activity 5.1: Launch an EC2 instance to host the Flask application.**

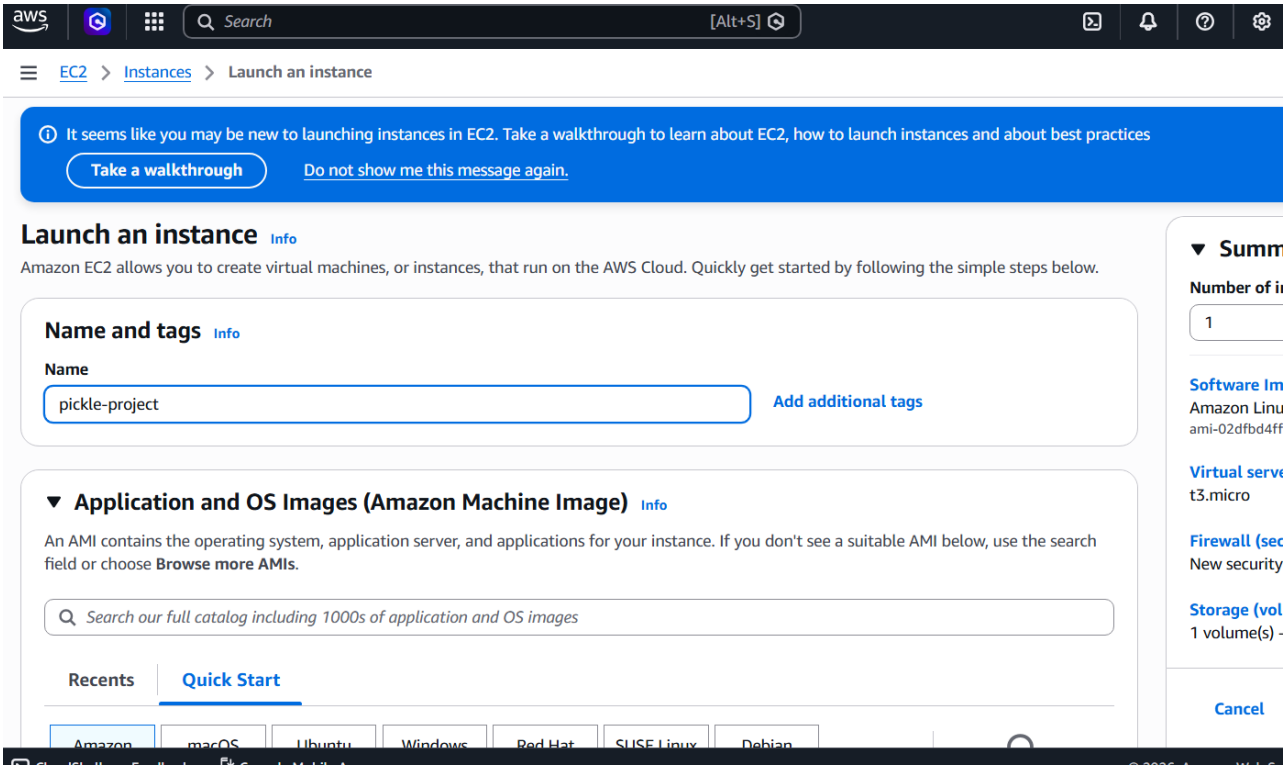
    - **Launch EC2 Instance**
        - In the AWS Console, navigate to EC2 and launch a new instance.

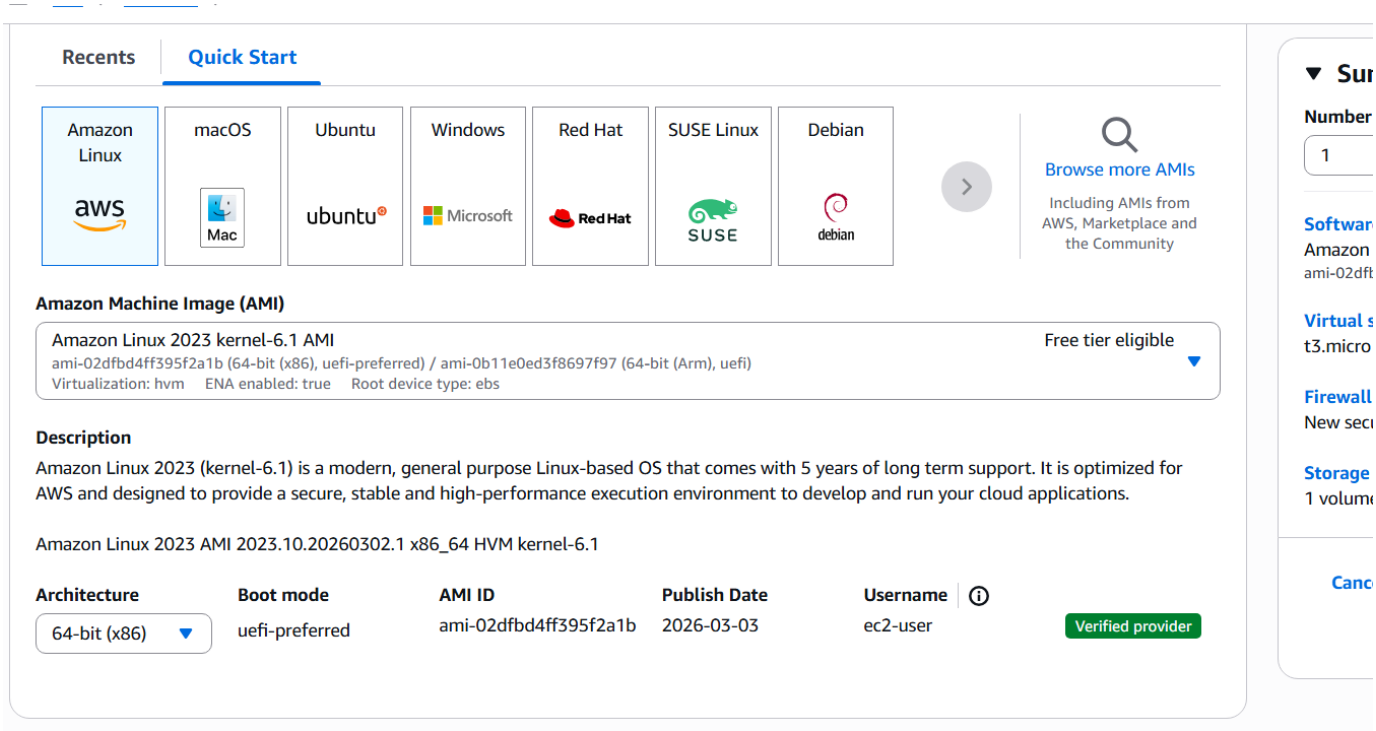


- Click on Launch instance to launch EC2 instance



- Choose Amazon Linux 2 or Ubuntu as the AMI and t2.micro as the instance type (free-tier eligible).





Create key pair

Key pair name
Key pairs allow you to connect to your instance securely.

InstantLibrary

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

- (•) **RSA** — RSA encrypted private and public key pair
- ( ) **ED25519** — ED25519 encrypted private and public key pair

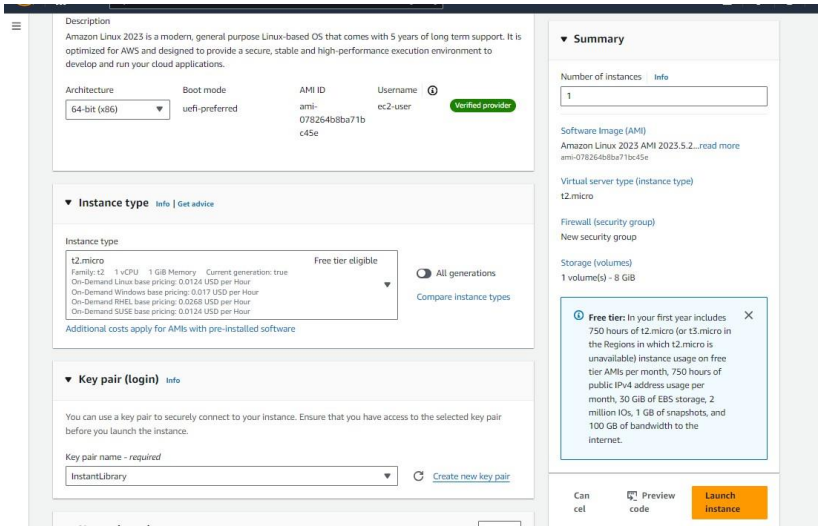
Private key file format

- (•) **.pem** — For use with OpenSSH
- ( ) **.ppk** — For use with PuTTY

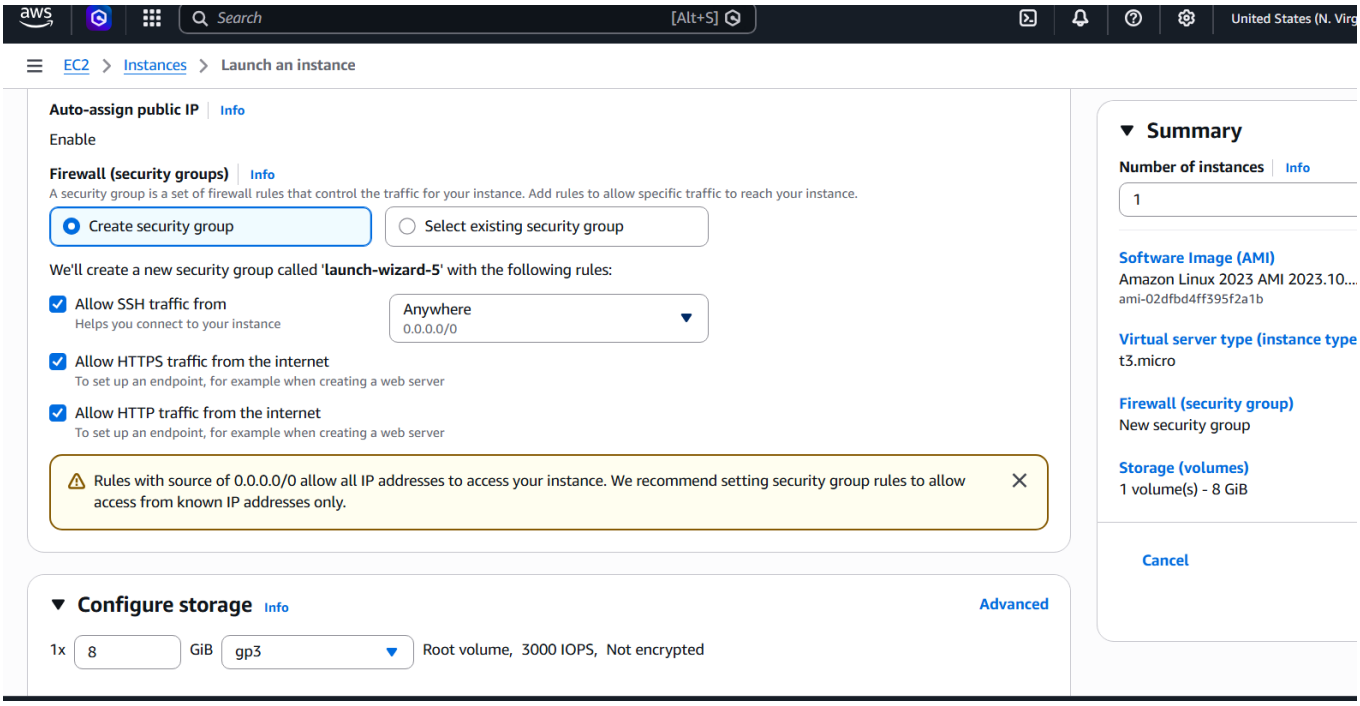
⚠ When prompted, store the private key in a secure and accessible location on your computer. **You will need it later to connect to your instance.** Learn more

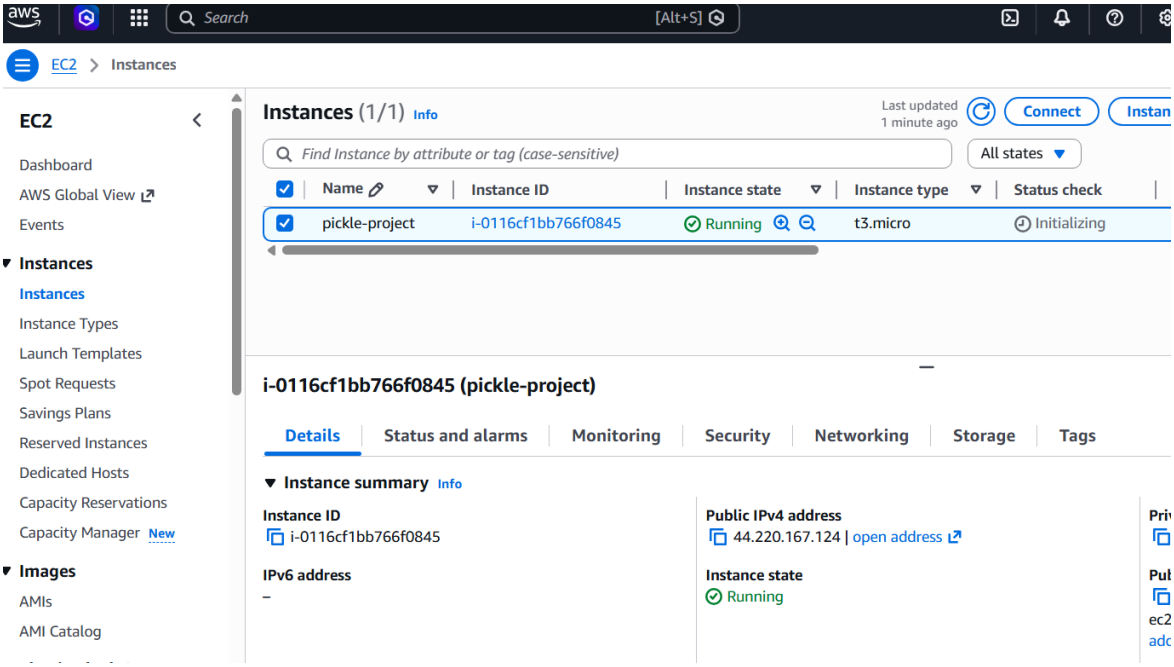
Cancel | **Create key pair**

InstantLibrary.pem



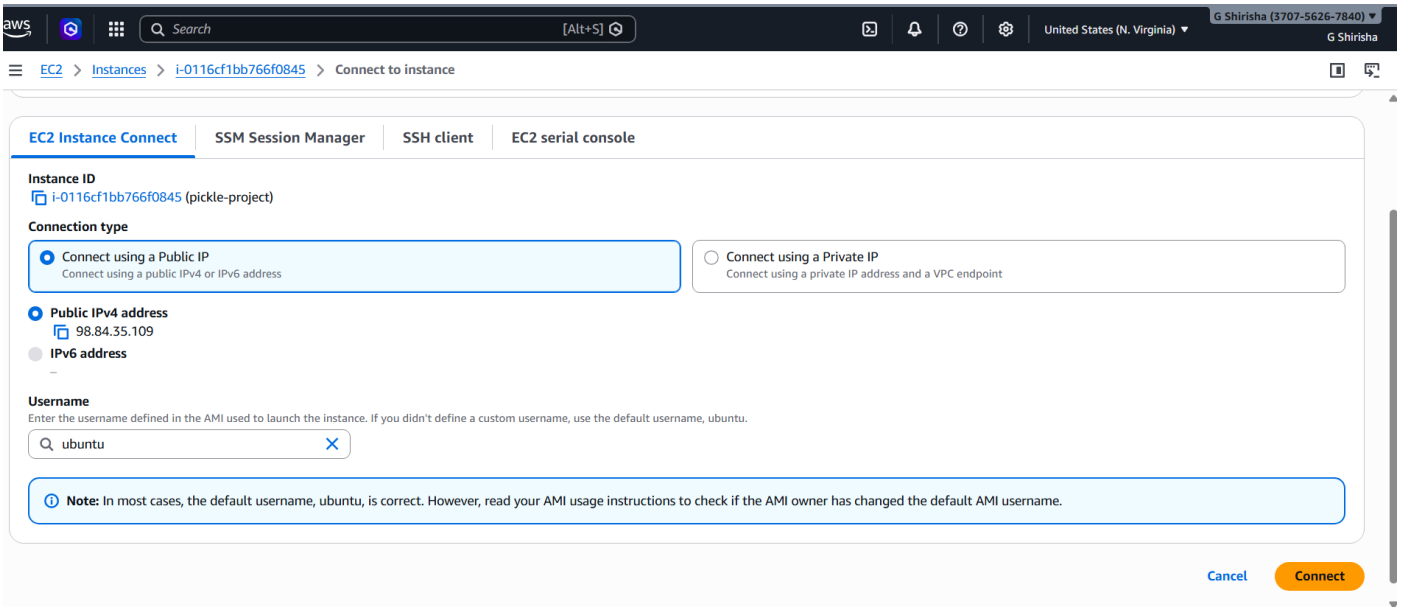
- **Activity 5.2:Configure security groups for HTTP, and SSH access.**





- To connect to EC2 using **EC2 Instance Connect**, start by ensuring that an **IAM role** is attached to your EC2 instance. You can do this by selecting your instance, clicking on **Actions**, then navigating to **Security** and selecting **Modify IAM Role** to attach the appropriate role. After the IAM role is connected, navigate to the **EC2** section in the **AWS Management Console**. Select the **EC2 instance** you wish to connect to. At the top of the **EC2 Dashboard**, click the **Connect** button. From the connection methods presented, choose **EC2 Instance Connect**. Finally, click **Connect** again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.

- Now connect the EC2 with the files



Milestone 6: Deployment on EC2

Activity 6.1: Install Software on the EC2 Instance

 # Update and install Python + pip

sudo apt update

sudo apt install python3-pip -y

sudo apt install git -y

sudo apt install python3-venv -y

python3 -m venv venv

source venv/bin/activate

Activity 6.2:Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

Run: 'git clone https://github.com/your-github-username/your-repository-name.git'

Note: change your-github-username and your-repository-name with your credentials

here: 'https://github.com/Shirisha-g08/homemade-main/tree/main'
This will download your project to the EC2 instance.

To navigate to the project directory, run the following command:

cd <your-folder>

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Run the Flask Application

git pull origin main (Only if you made changes in git and want to reflect them in the EC2 terminal.)

 # Install requirements

pip install flask boto3

Launch the Flask app:

sudo -E venv/bin/python3 app.py

Verify the Flask app is running:

http://your-ec2-public-ip

- Run the Flask app on the EC2 instance

```
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in ./venv/lib/python3.12/si
Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in ./venv/lib/python3.12/sit
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/site-packages (from p
(venv) ubuntu@ip-172-31-1-192:~/homemade-main/homemade-main$ python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.31.1.192:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 215-327-013
```

i-0116cf1bb766f0845 (pickle-project)

PublicIPs: 98.84.35.109 PrivateIPs: 172.31.1.192

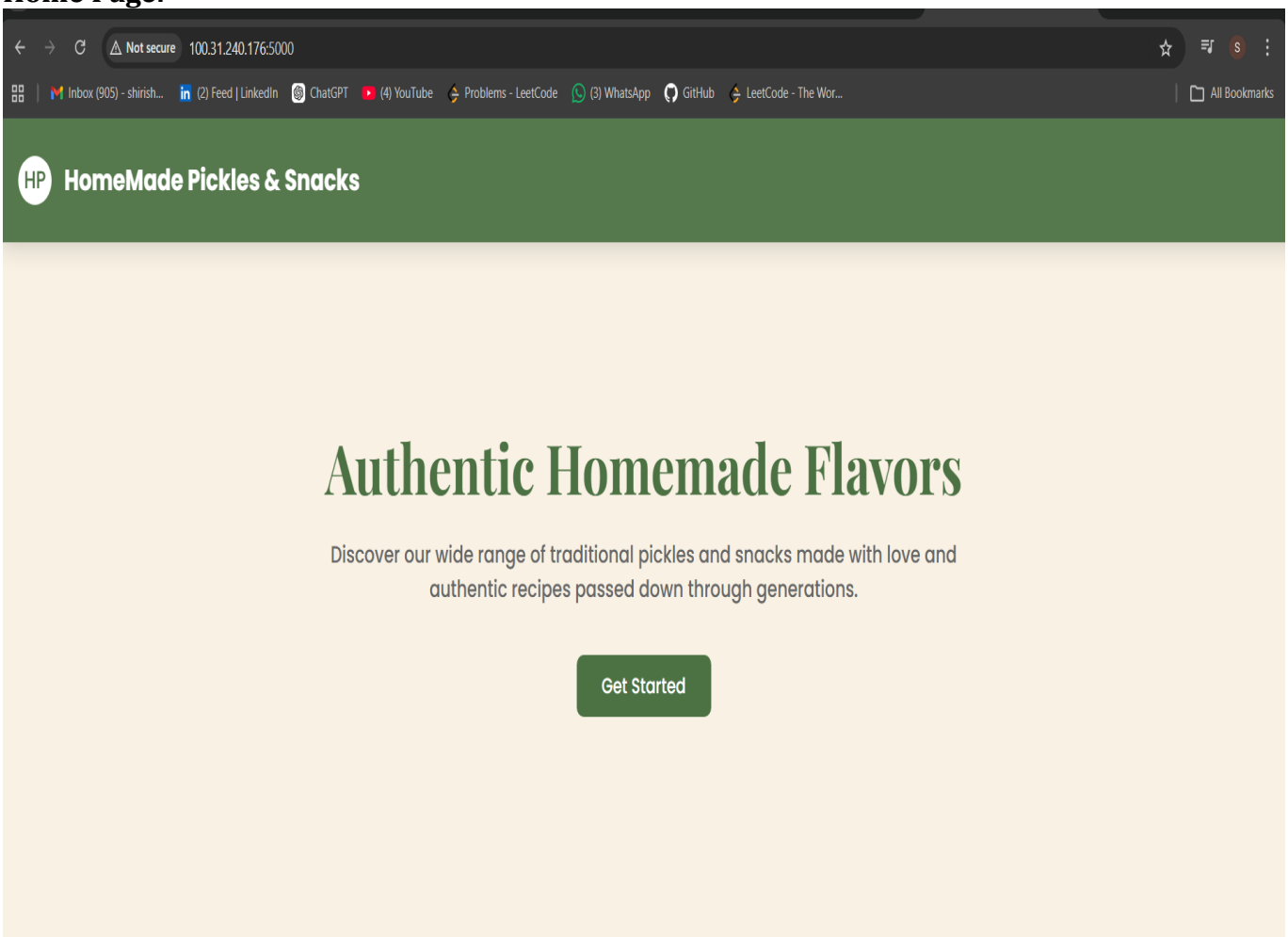
Access the website through:

PublicIPs: http://98.84.35.109:5000/

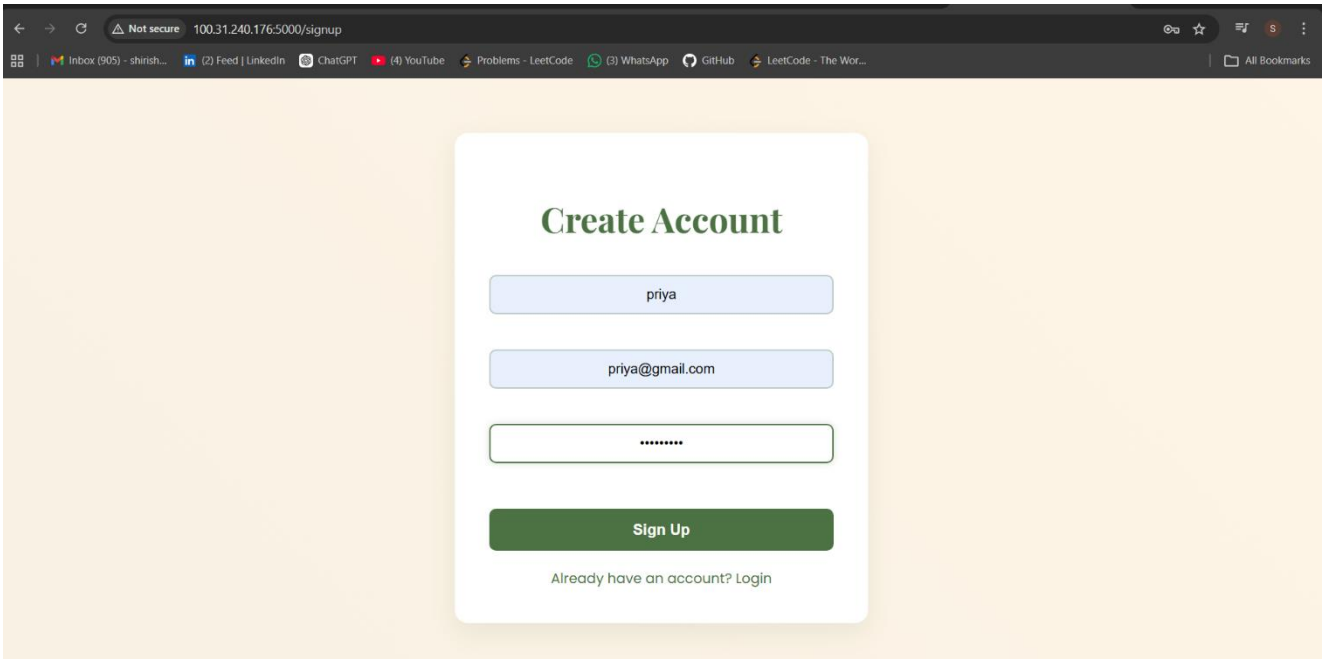
Milestone 7: Testing and Deployment

- **Activity 7.1: Conduct functional testing to verify user registration, login, order requests.**

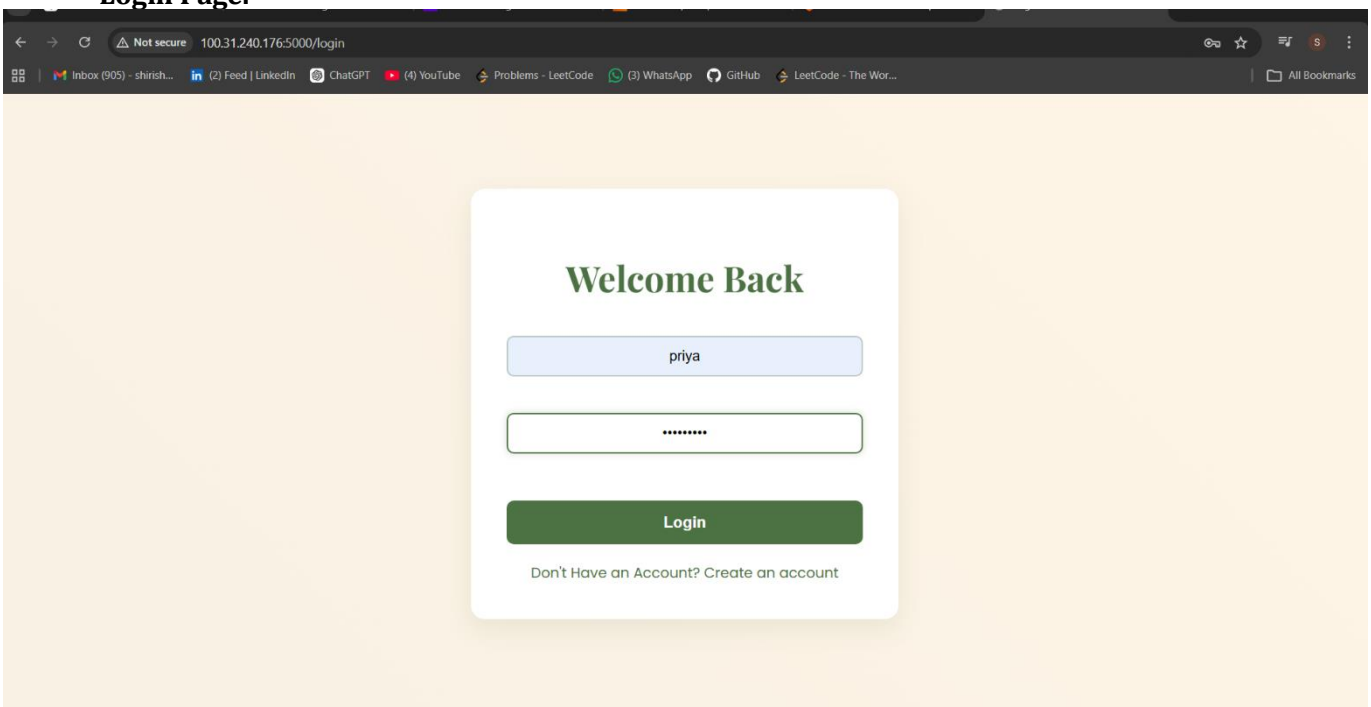
Home Page:



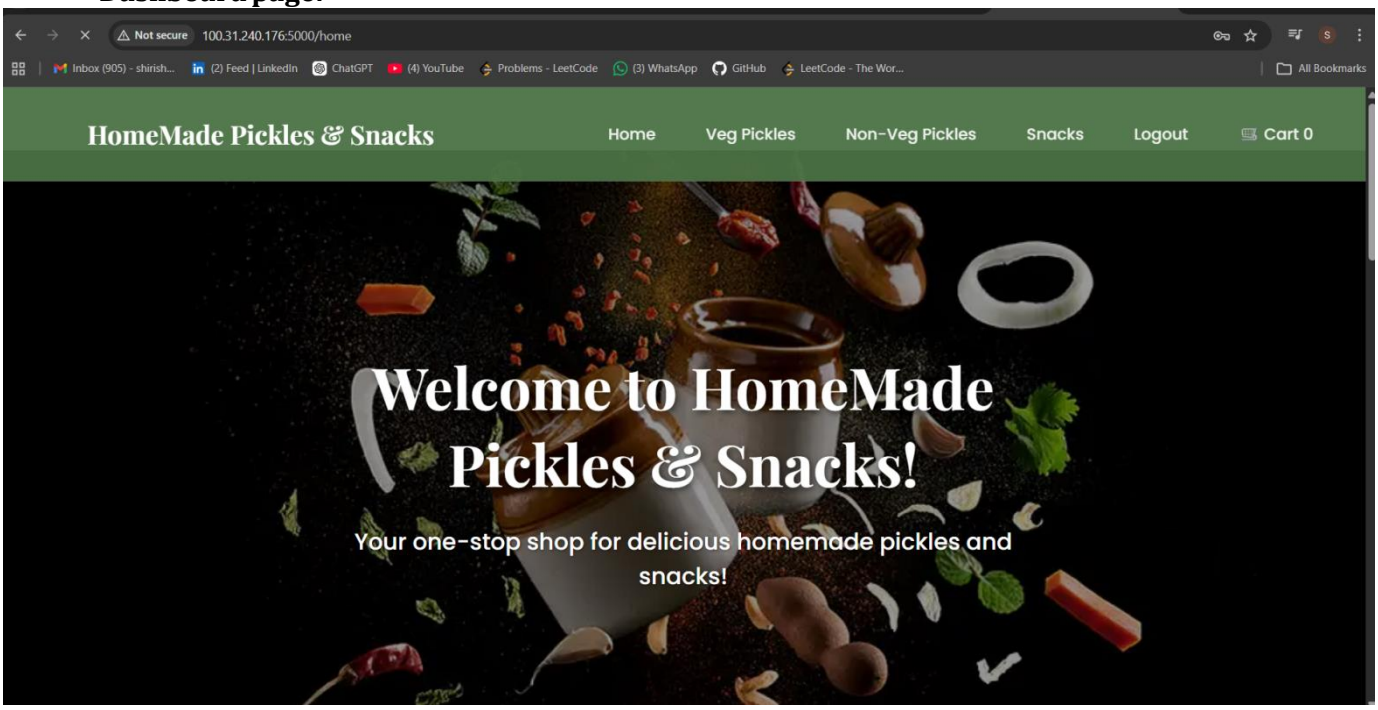
Register Page:



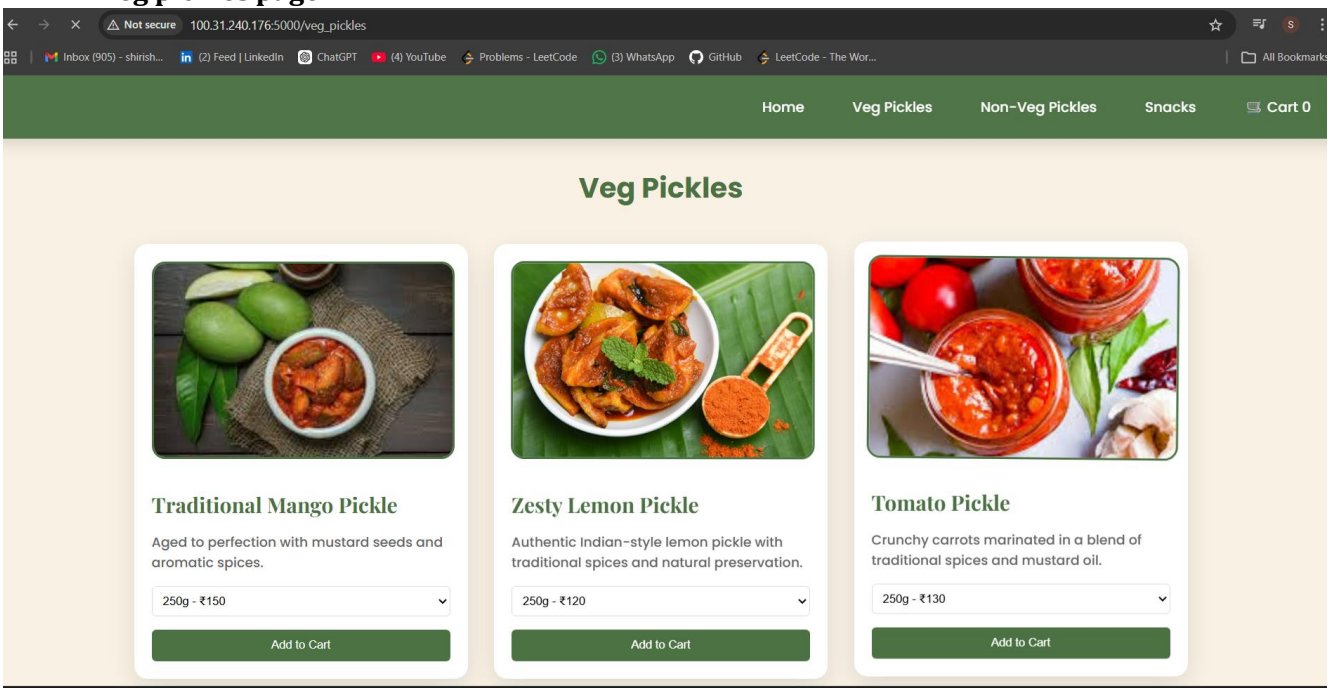
Login Page:



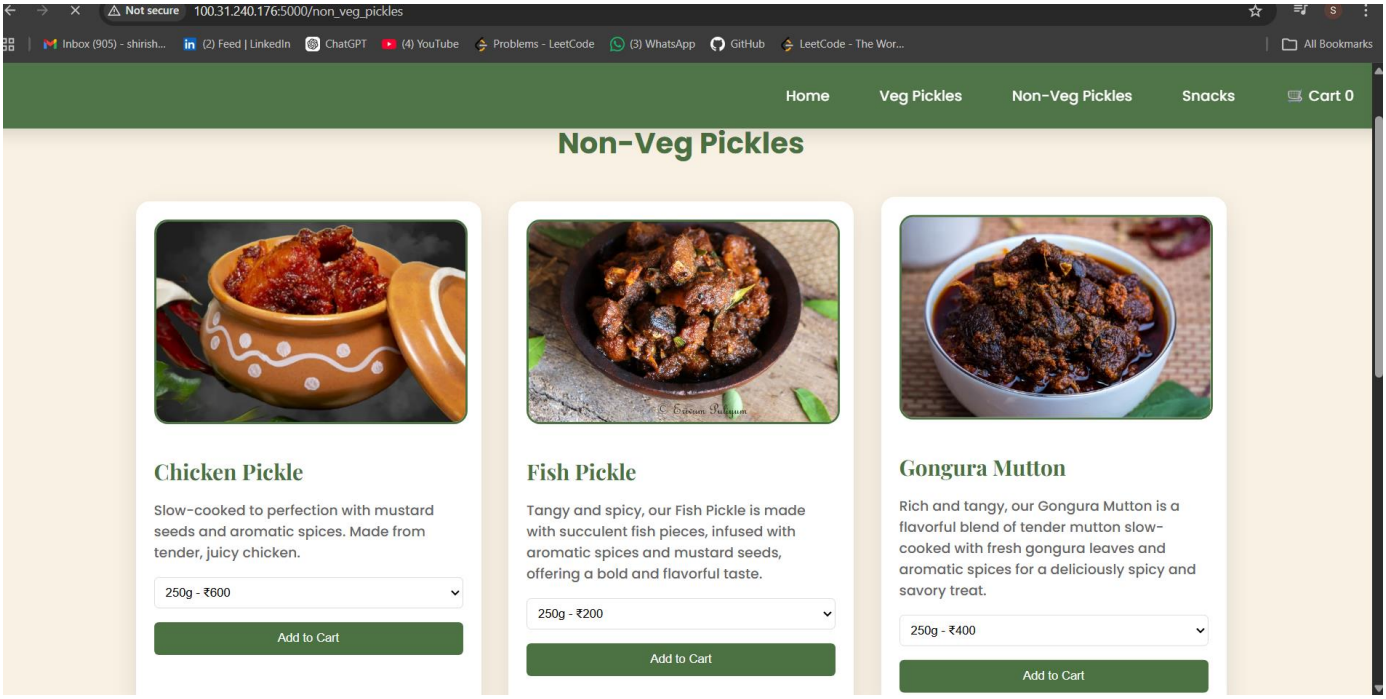
Dashboard page:



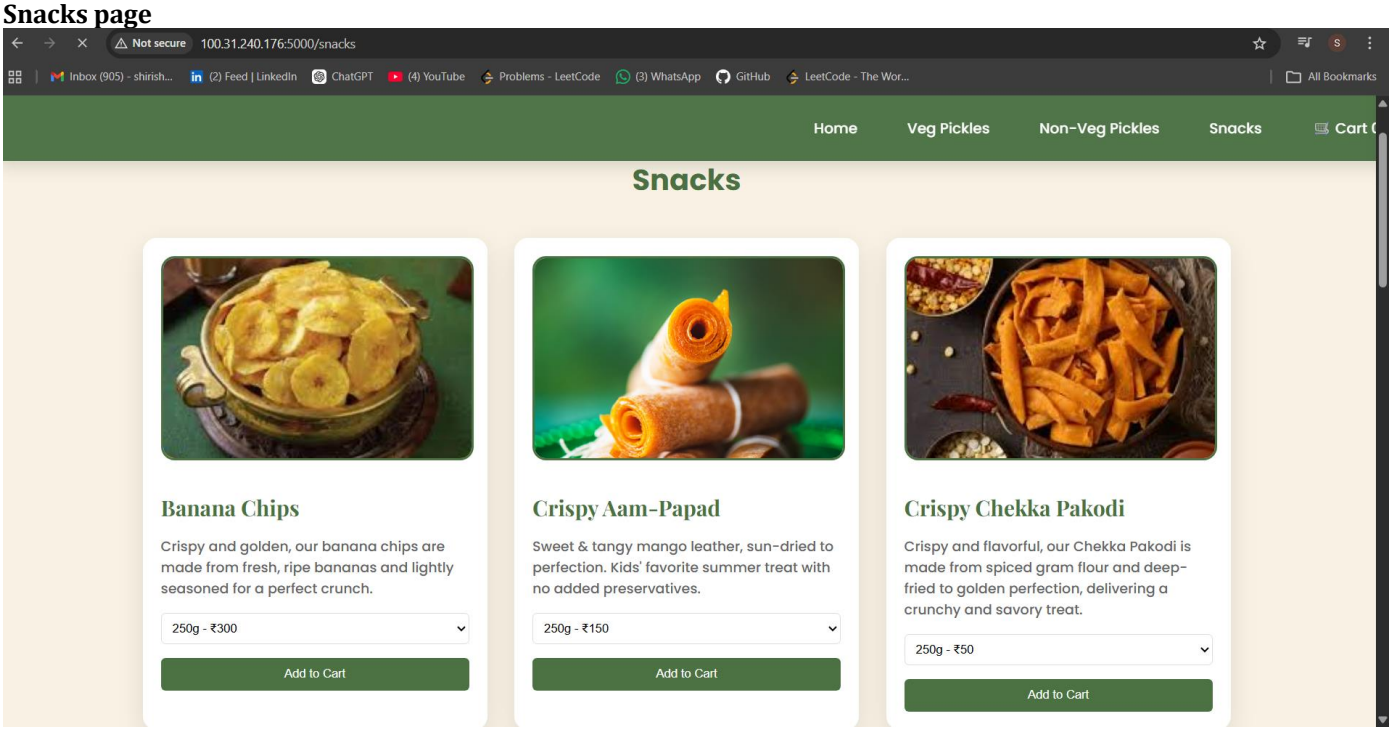
Veg pickles page:



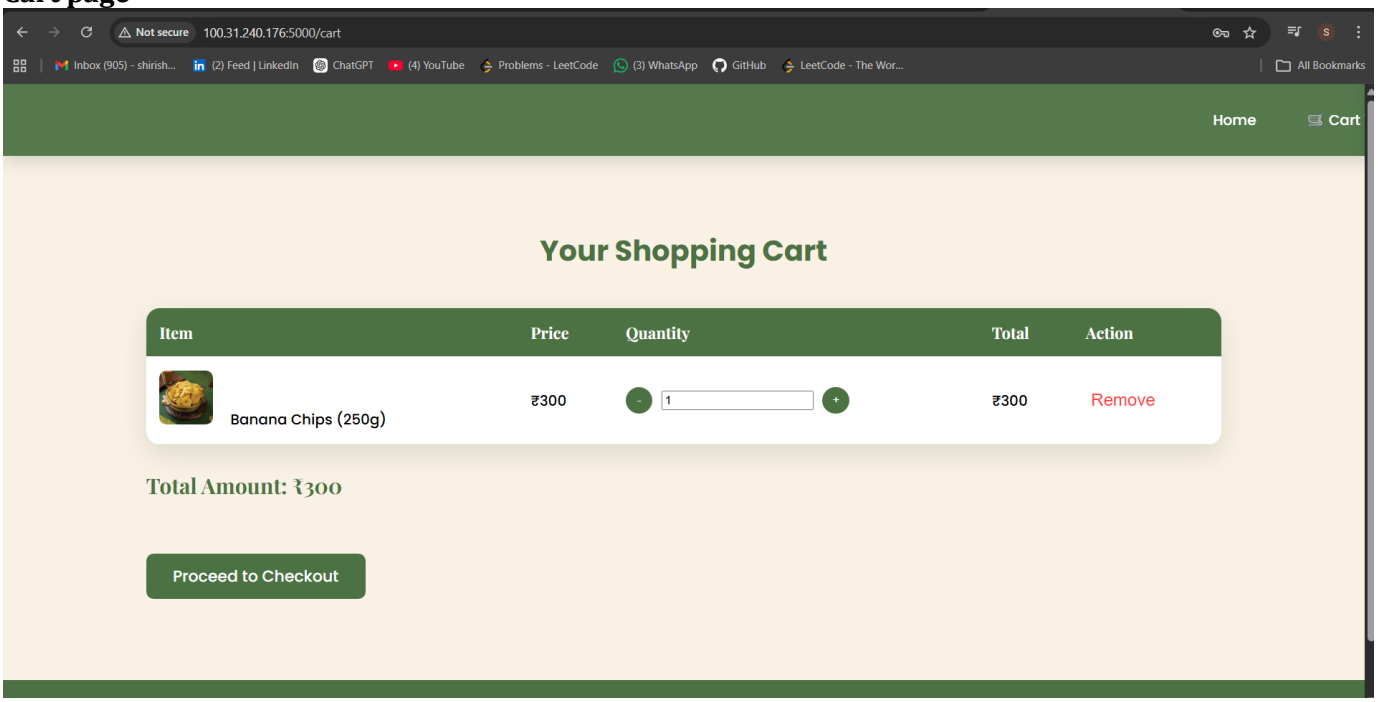
Non veg pickles page



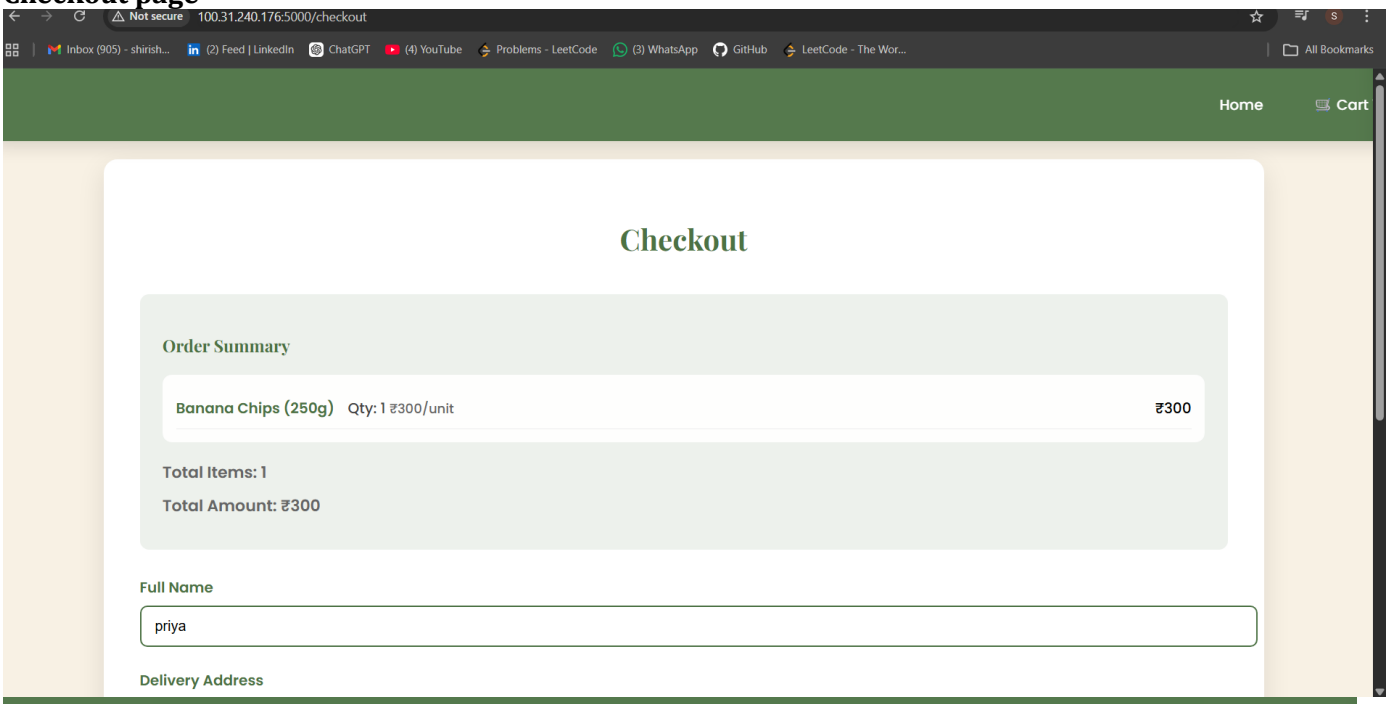
Snacks page

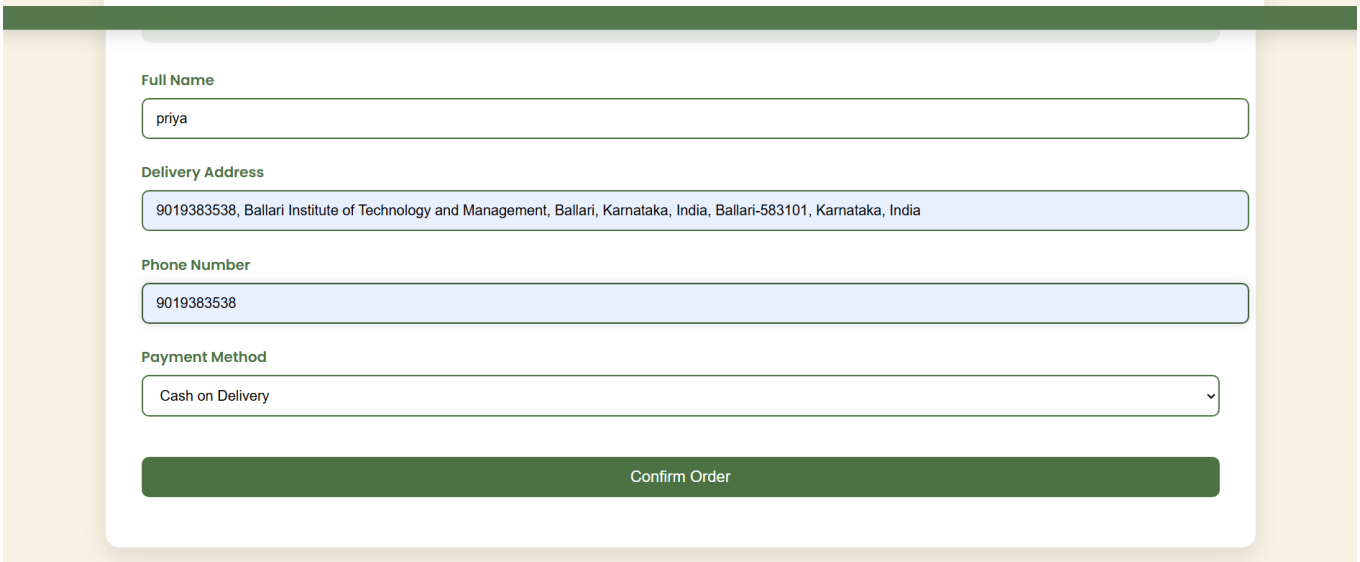


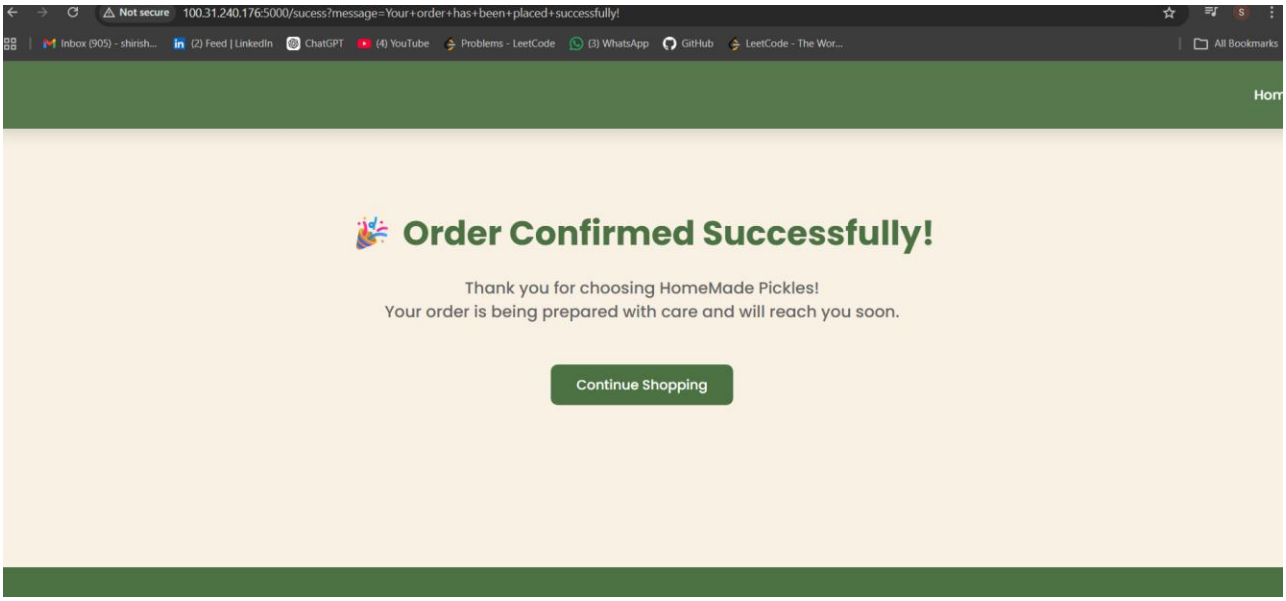
Cart page



Checkout page

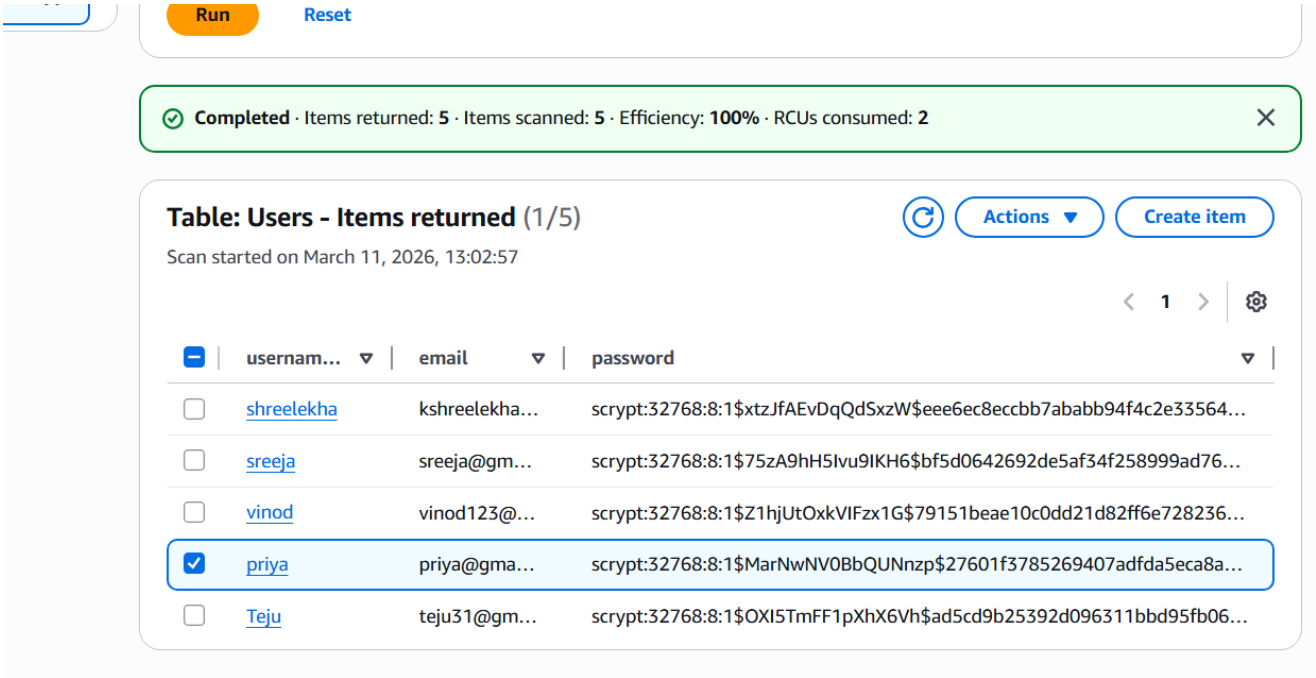





Not secure 100.31.240.176:5000/sucess?message=Your+order+has+been+placed+successfully!
Inbox (905) - shirish...
(2) Feed | LinkedIn
ChatGPT
(4) YouTube
Problems - LeetCode
(3) WhatsApp
GitHub
LeetCode - The Wor...
All Bookmarks
Hom
Order Confirmed Successfully!
Thank you for choosing HomeMade Pickles!
Your order is being prepared with care and will reach you soon.
Continue Shopping

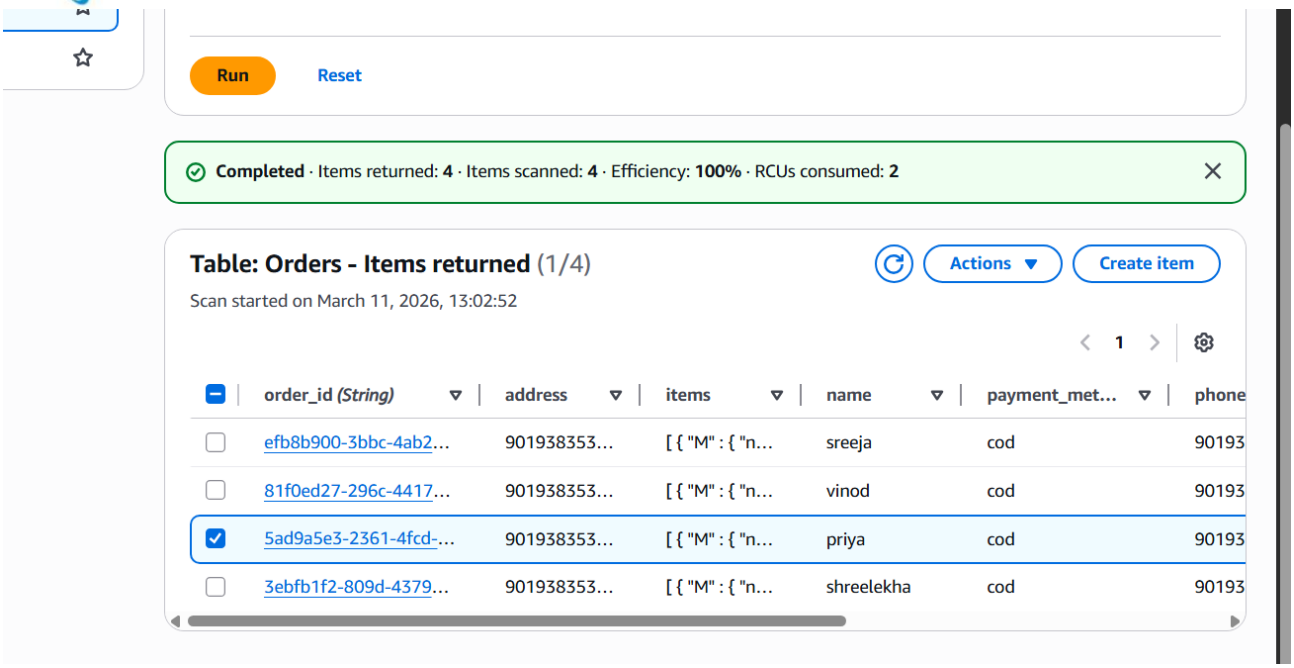
Dynamodb Database updations :

1. **users table :**



Description: Store and manage user credentials, login counts, and names in the `Users` table on DynamoDB for authentication and tracking purposes.

2. **orders table :**



Description: Use the order table in DynamoDB to store user order details.

Conclusion:

The *HomeMade Pickles & Snacks: Taste the Best* platform has been successfully developed as a cloud-based web application that allows users to conveniently browse and order authentic homemade pickles and snacks through an easy-to-use interface. The system provides essential features such as user registration, login, product browsing, cart management, and order placement, making the overall shopping experience simple and efficient for customers.

The application is designed using modern web technologies and cloud services for hosting and database management, which ensures reliable performance and efficient data handling. The backend manages product information, user details, and order records while maintaining secure and organized storage of data.

Overall, the project demonstrates how cloud-based platforms can be used to build a practical e-commerce system for homemade food products. It helps small-scale homemade food businesses reach a wider audience while providing customers with convenient access to traditional homemade pickles and snacks.